

===== README.md =====

Manish Mishra Academy (MMA)

Full-stack coaching academy website with public content, role-based portals, academic resources, fee management, and Razorpay-ready sandbox payments.

Requirements

- Node.js 20 LTS or newer
- npm 10 or newer

Technology

- Frontend: React, Vite, TypeScript, Tailwind CSS, React Router
- Backend: Node.js, Express, TypeScript, Zod
- Database: SQLite with Prisma ORM
- Authentication: JWT in HTTP-only cookie, backend role and permission checks
- Payments: Razorpay REST API and signed webhook verification

Installation

```
'''bash
npm install
npm --prefix client install
npm --prefix server install
copy .env.example .env
'''
```

Never commit '.env', payment secrets, private uploads, or local database files.

Environment

Set these values in the root '.env':

```
'''env
PORT=4000
CLIENT_URL=http://localhost:5173
DATABASE_URL="file:./dev.db"
JWT_SECRET=use-a-long-random-secret
ADMIN_EMAIL=replace-with-admin-email
ADMIN_PASSWORD=replace-with-admin-password
PAYMENT_KEY_ID=
PAYMENT_KEY_SECRET=
PAYMENT_WEBHOOK_SECRET=
PAYMENT_SANDBOX=true
'''
```

Payment secrets are read only by the server and are never sent to the frontend.

Database

```
'''bash
npm run db:generate
npm run db:push
npm run db:seed
npm --prefix server exec tsx -- prisma/seed-part2.ts
'''
```

For a clean local development database only:

```
'''bash
npm --prefix server exec prisma db push -- --force-reset
```

```
npm run db:seed
npm --prefix server exec tsx -- prisma/seed-part2.ts
'''
```

The seed creates clearly marked development accounts and sample academic assignments. It does not invent real MMA fee structures.

Development

```
'''bash
npm run dev
'''
```

```
- Frontend: http://localhost:5173
- Backend: http://localhost:4000
```

Production

```
'''bash
npm run build
npm --prefix server start
'''
```

Configure HTTPS, a production database, secure cookies, and real Razorpay webhook settings before launch.

Authentication and authorization

Roles are 'HEAD', 'SUPERVISOR', 'TEACHER', and 'STUDENT'. There is no public student or teacher registration. The backend enforces record ownership, teacher class/subject assignments, student enrollments, and supervisor permissions.

Payment

Set Razorpay test credentials in '.env' and keep 'PAYMENT_SANDBOX=true' during development. The backend verifies payment signatures and webhooks, uses transactions for balance updates, prevents overpayment, and creates receipts only for verified successful payments. The no-credential development mode creates internal sandbox orders for workflow testing; it is not a real payment gateway.

Testing

```
'''bash
npm --prefix server exec tsc -- -p tsconfig.json
npm --prefix client exec tsc -- -b tsconfig.app.json
npm run build
'''
```

The security checks cover invalid signatures, failed payments, duplicate verification, student fee privacy, teacher assignment restrictions, private downloads, and supervisor payment permission denial.

Real MMA information still required

- Official address, phone, email, and WhatsApp details
- Approved logo and brand assets
- Real faculty names and assignments
- Actual subjects, batches, timetable, and photographs
- Approved fee structures and policies
- Production Razorpay account and webhook URL
- Privacy, refund, cancellation, and terms policies
- Production domain for canonical URLs and sitemap entries

```
==== .env.example ====
PORT=4000
```

```

JWT_SECRET=replace_with_secure_secret
CLIENT_URL=http://localhost:5173
DATABASE_URL="file:./dev.db"
ADMIN_EMAIL=replace-with-admin-email
ADMIN_PASSWORD=replace-with-admin-password
PAYMENT_KEY_ID=
PAYMENT_KEY_SECRET=
PAYMENT_WEBHOOK_SECRET=
PAYMENT_SANDBOX=true

===== .gitignore =====
node_modules/
.env
**/.env
.env.*
!.env.example
client/dist/
server/dist/
server/prisma/dev.db
server/prisma/dev.db-journal
server/private-uploads/
.DS_Store
npm-debug.log*
coverage/
.vite/

===== package.json =====
{
  "name": "mma-academy",
  "private": true,
  "version": "1.0.0",
  "scripts": {
    "dev": "concurrently \"npm --prefix client run dev\" \"npm --prefix server run dev\"",
    "build": "npm --prefix client run build && npm --prefix server run build",
    "db:generate": "npm --prefix server run db:generate",
    "db:push": "npm --prefix server run db:push",
    "db:seed": "npm --prefix server run db:seed"
  },
  "devDependencies": {
    "concurrently": "^9.1.2"
  }
}

===== client/index.html =====
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Manish Mishra Academy (MMA) provides focused academic coaching for Classes
7 to 12 with personal attention, modern classrooms, and a disciplined learning environment." />
    <meta property="og:title" content="Manish Mishra Academy (MMA) | Coaching for Classes 7?12" />
    <meta property="og:description" content="Learn Better. Think Smarter. Achieve More." />
    <meta property="og:type" content="website" />
    <meta name="robots" content="index,follow" />
    <title>Manish Mishra Academy (MMA) | Coaching for Classes 7?12</title>
  </head>
  <body class="bg-slate-50">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>

```

===== client/public/robots.txt =====

```
User-agent: *
Allow: /
Disallow: /login
Disallow: /admin/
Disallow: /student/
Disallow: /teacher/
```

===== client/public/sitemap.xml =====

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>http://localhost:5173/</loc></url>
  <url><loc>http://localhost:5173/about</loc></url>
  <url><loc>http://localhost:5173/courses</loc></url>
  <url><loc>http://localhost:5173/facilities</loc></url>
  <url><loc>http://localhost:5173/timetable</loc></url>
  <url><loc>http://localhost:5173/faq</loc></url>
  <url><loc>http://localhost:5173/contact</loc></url>
</urlset>
```

===== client/src/App.tsx =====

```
import { Route, Routes } from 'react-router-dom';
import Layout from './components/Layout';
import HomePage from './pages/HomePage';
import AboutPage from './pages/AboutPage';
import CoursesPage from './pages/CoursesPage';
import Classes710Page from './pages/Classes710Page';
import Classes1112Page from './pages/Classes1112Page';
import FacilitiesPage from './pages/FacilitiesPage';
import TimetablePage from './pages/TimetablePage';
import StudyEnvironmentPage from './pages/StudyEnvironmentPage';
import FaqPage from './pages/FaqPage';
import ContactPage from './pages/ContactPage';
import LoginPage from './pages/LoginPage';
import PortalPage from './pages/PortalPage';
import ResourcesPage from './pages/ResourcesPage';
import AcademicListPage from './pages/AcademicListPage';
import FeesPage from './pages/FeesPage';
import NotFoundPage from './pages/NotFoundPage';

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<HomePage />} />
      <Route path="about" element={<AboutPage />} />
      <Route path="courses" element={<CoursesPage />} />
      <Route path="courses/classes-7-10" element={<Classes710Page />} />
      <Route path="courses/classes-11-12" element={<Classes1112Page />} />
      <Route path="facilities" element={<FacilitiesPage />} />
      <Route path="timetable" element={<TimetablePage />} />
      <Route path="study-environment" element={<StudyEnvironmentPage />} />
      <Route path="faq" element={<FaqPage />} />
      <Route path="contact" element={<ContactPage />} />
      <Route path="login" element={<LoginPage />} />
      <Route path="admin/dashboard" element={<PortalPage />} />
      <Route path="teacher/dashboard" element={<PortalPage />} />
      <Route path="student/dashboard" element={<PortalPage />} />
      <Route path="student/resources" element={<ResourcesPage mode="student" />} />
      <Route path="student/tests" element={<AcademicListPage kind="tests" role="student" />} />
      <Route path="student/timetable" element={<AcademicListPage kind="timetable" role="student" />} />
    </Routes>
  );
}
```

```

    <Route path="student/fees" element={<FeesPage mode="student" />} />
    <Route path="teacher/resources" element={<ResourcesPage mode="teacher" />} />
    <Route path="teacher/tests" element={<AcademicListPage kind="tests" role="teacher" />} />
    <Route path="teacher/timetable" element={<AcademicListPage kind="timetable" role="teacher" />} />
    <Route path="admin/tests" element={<AcademicListPage kind="tests" role="admin" />} />
    <Route path="admin/fees" element={<FeesPage mode="admin" />} />
    <Route path="admin/payments" element={<FeesPage mode="admin" />} />
    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
);
}

```

===== client/src/main.tsx =====

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './styles.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
);

```

===== client/src/styles.css =====

```

@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color: #0f172a;
  background-color: #f8fafc;
  font-family: Inter, 'Segoe UI', sans-serif;
  line-height: 1.5;
  font-weight: 400;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  min-width: 320px;
  background: linear-gradient(180deg, #f8fbff 0%, #eef6ff 100%);
}

* {
  box-sizing: border-box;
}

img {
  max-width: 100%;
  display: block;
}

```

```

}

button, input, select, textarea {
  font: inherit;
}

@layer components {
  .container-shell {
    @apply mx-auto max-w-7xl px-4 sm:px-6 lg:px-8;
  }

  .section-title {
    @apply text-3xl font-bold tracking-tight text-slate-900 md:text-4xl;
  }

  .section-subtitle {
    @apply mt-4 max-w-2xl text-base text-slate-600 md:text-lg;
  }

  .card-surface {
    @apply rounded-2xl border border-slate-200 bg-white shadow-[0_12px_35px_rgba(15,23,42,0.06)];
  }

  .primary-button {
    @apply inline-flex items-center justify-center rounded-full bg-sky-600 px-6 py-3 text-sm font-semibold text-white shadow-lg shadow-sky-200 transition hover:-translate-y-0.5 hover:bg-sky-700 focus-visible:outline focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-sky-600;
  }

  .secondary-button {
    @apply inline-flex items-center justify-center rounded-full border border-slate-200 bg-white px-6 py-3 text-sm font-semibold text-slate-800 transition hover:-translate-y-0.5 hover:border-sky-200 hover:text-sky-700;
  }
}

===== client/src/components/Layout.tsx =====
import { Menu, X } from 'lucide-react';
import { useState } from 'react';
import { NavLink, Outlet } from 'react-router-dom';

const navItems = [
  { label: 'Home', to: '/' },
  { label: 'About', to: '/about' },
  { label: 'Courses', to: '/courses' },
  { label: 'Facilities', to: '/facilities' },
  { label: 'Timetable', to: '/timetable' },
  { label: 'Study Environment', to: '/study-environment' },
  { label: 'FAQ', to: '/faq' },
  { label: 'Contact', to: '/contact' },
  { label: 'Login', to: '/login' },
];

export default function Layout() {
  const [mobileOpen, setMobileOpen] = useState(false);

  return (
    <div className="min-h-screen bg-slate-50 text-slate-800">
      <header className="sticky top-0 z-50 border-b border-slate-200/80 bg-white/85 backdrop-blur-md">
        <div className="container-shell flex items-center justify-between py-4">
          <NavLink to="/" className="flex items-center gap-3">
            <div className="flex h-11 w-11 items-center justify-center rounded-xl bg-sky-600 font-black text-lg text-white shadow-md shadow-sky-200">M</div>

```

```

    <div>
      <div className="text-sm font-bold uppercase tracking-[0.2em] text-sky-700">MMA</div>
      <div className="text-xs text-slate-500">Academy</div>
    </div>
  </NavLink>

  <nav className="hidden items-center gap-6 lg:flex">
    {navItems.map((item) => (
      <NavLink
        key={item.to}
        to={item.to}
        className={({ isActive }) =>
          'text-sm font-medium transition ${isActive ? 'text-sky-700' : 'text-slate-600 hover:text-sky-7
00'}'
        }
      >
        {item.label}
      </NavLink>
    ))}
  </nav>

  <div className="hidden lg:block">
    <NavLink to="/contact" className="primary-button">
      Enquire Now
    </NavLink>
  </div>

  <button
    type="button"
    className="rounded-lg border border-slate-200 p-2 text-slate-700 lg:hidden"
    onClick={() => setMobileOpen((value) => !value)}
    aria-label="Toggle navigation"
  >
    {mobileOpen ? <X size={20} /> : <Menu size={20} />}
  </button>
</div>

{mobileOpen && (
  <div className="border-t border-slate-200 bg-white lg:hidden">
    <div className="container-shell flex flex-col gap-2 py-4">
      {navItems.map((item) => (
        <NavLink
          key={item.to}
          to={item.to}
          onClick={() => setMobileOpen(false)}
          className={({ isActive }) =>
            'rounded-lg px-3 py-2 text-sm font-medium ${isActive ? 'bg-sky-50 text-sky-700' : 'text-slat
e-700 hover:bg-slate-100'}'
          }
        >
          {item.label}
        </NavLink>
      ))}
    </div>
  </div>
)}
</header>

<main>
  <Outlet />
</main>

```

```

<footer className="mt-16 border-t border-slate-200 bg-slate-950 text-slate-200">
  <div className="container-shell grid gap-10 py-12 md:grid-cols-3">
    <div>
      <div className="mb-4 text-xl font-bold text-white">Manish Mishra Academy</div>
      <p className="text-sm text-slate-300">
        Focused academic coaching for Classes 7th to 12th with personal guidance, disciplined study habits
, and modern learning support.
      </p>
    </div>
    <div>
      <h3 className="mb-4 text-sm font-semibold uppercase tracking-[0.2em] text-sky-300">Quick Links</h3>
      <ul className="space-y-2 text-sm text-slate-300">
        <li><NavLink to="/about">About</NavLink></li>
        <li><NavLink to="/courses">Courses</NavLink></li>
        <li><NavLink to="/facilities">Facilities</NavLink></li>
        <li><NavLink to="/contact">Contact</NavLink></li>
      </ul>
    </div>
    <div>
      <h3 className="mb-4 text-sm font-semibold uppercase tracking-[0.2em] text-sky-300">Contact</h3>
      <ul className="space-y-2 text-sm text-slate-300">
        <li>Placeholder phone number</li>
        <li>Placeholder email address</li>
        <li>Placeholder academy address</li>
      </ul>
    </div>
  </div>
</footer>
</div>
  );
}

```

```

===== client/src/pages/AboutPage.tsx =====

```

```

const milestones = [
  'Concept-focused teaching for Classes 7th to 12th',
  'Regular doubt support and teacher guidance',
  'Academic monitoring and practice-based improvement',
  'Study environment designed around focus and discipline',
];

```

```

export default function AboutPage() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="grid gap-10 lg:grid-cols-[0.9fr_1.1fr] lg:items-center">
        <div>
          <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">About us</p>
          <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">About Manish Mishra Academy</h1>
          <p className="mt-5 text-lg text-slate-600">
            MMA is designed for students who want clarity, consistency and confidence in academics. We believe s
trong fundamentals matter more than rote learning, which is why conceptual learning and regular practice remain
at the centre of our approach.
          </p>
          <p className="mt-5 text-slate-600">
            Our system encourages students to understand ideas deeply, ask questions without hesitation, and imp
rove gradually through regular assessment and guided support.
          </p>
        </div>
        <div className="card-surface overflow-hidden p-3">
          
        </div>
      </div>
    </div>
  );
}

```

```

    />
  </div>
</div>

<div className="mt-16 grid gap-6 md:grid-cols-2 xl:grid-cols-4">
  {milestones.map((item, index) => (
    <div key={item} className="card-surface p-6">
      <div className="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">0{index + 1}</di
v>
      <p className="text-lg font-semibold text-slate-900">{item}</p>
    </div>
  ))}
</div>
</div>
);
}

```

```

===== client/src/pages/AcademicListPage.tsx =====
import { useEffect, useState } from 'react';

```

```

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000';

```

```

type RecordItem = Record<string, string | undefined> & { id: string };

```

```

export default function AcademicListPage({ kind, role }: { kind: 'tests' | 'timetable'; role: 'student' | 'teacher' | 'admin' }) {
  const [items, setItems] = useState<RecordItem[]>([]);
  const [error, setError] = useState('');
  const endpoint = '/api/${role} === 'admin' ? 'admin' : role}/${kind}';

```

```

  useEffect(() => { fetch(`${API_URL}${endpoint}`, { credentials: 'include' }).then(async (response) => { const
payload = await response.json(); if (!response.ok) throw new Error(payload.message || 'Unable to load academic r
ecords. '); setItems(payload.data || []); }).catch((loadError: unknown) => setError(loadError instanceof Error ?
loadError.message : 'Unable to load academic records. ')); }, [endpoint]);

```

```

  return <div className="container-shell py-12"><p className="text-sm font-semibold uppercase tracking-[0.2em] t
ext-sky-700">Academic portal</p><h1 className="mt-3 text-4xl font-bold text-slate-900">{kind === 'tests' ? 'Test
s' : 'Timetable'}</h1><p className="mt-3 text-slate-600">Database-driven records available to your account.</p>{
error ? <p className="card-surface mt-8 p-6 text-red-700">{error}</p> : <div className="mt-8 grid gap-4 md:grid-
cols-2">{items.map((item) => <article className="card-surface p-6" key={item.id}><h2 className="text-xl font-bol
d text-slate-900">{item.title || `${item.day} ? ${item.subject || 'Class session'}`}</h2><p className="mt-3 text-
-sm text-slate-600">{kind === 'tests' ? `${item.className} ? ${item.subject} ? ${item.topic}` : `${item.classLev
el} ? ${item.subject || 'Class session'} ? ${item.startTime} - ${item.endTime}`}</p><p className="mt-2 text-sm t
ext-slate-500">{kind === 'tests' ? `${item.date} at ${item.time} ? ${item.status}` : `${item.day}${item.batch} ?
' ? ${item.batch}` : ''}${item.room ? ' ? ${item.room}' : ''}}</p>{item.instructions && <p className="mt-3 text-
-sm text-slate-600">{item.instructions}</p></article>}}</div>{!items.length && !error && <p className="mt-8 te
xt-slate-600">No records are available yet.</p></div>;
}

```

```

===== client/src/pages/Classes710Page.tsx =====

```

```

const classes = [
  {
    title: 'Class 7',
    subjects: ['Mathematics', 'Science', 'English', 'Social Science', 'Hindi'],
    details: ['Concept building', 'Practice', 'Homework support', 'Regular tests', 'Doubt solving', 'Academic gu
idance'],
  },
  {
    title: 'Class 8',
    subjects: ['Mathematics', 'Science', 'English', 'Social Science', 'Hindi'],
    details: ['Concept building', 'Practice', 'Homework support', 'Regular tests', 'Doubt solving', 'Academic gu
idance'],
  },

```

```

    },
    {
      title: 'Class 9',
      subjects: ['Mathematics', 'Science', 'English', 'Social Science', 'Hindi'],
      details: ['Concept building', 'Practice', 'Homework support', 'Regular tests', 'Doubt solving', 'Academic guidance'],
    },
    {
      title: 'Class 10',
      subjects: ['Mathematics', 'Science', 'English', 'Social Science', 'Hindi'],
      details: ['Concept building', 'Practice', 'Homework support', 'Regular tests', 'Doubt solving', 'Academic guidance'],
    },
  ],
};

```

```

export default function Classes710Page() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="mb-10 text-center">
        <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Classes 7?10</p>
        <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Complete academic coaching</h1>
      </div>

      <div className="grid gap-6 lg:grid-cols-2">
        {classes.map((course) => (
          <div key={course.title} className="card-surface p-8">
            <div className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">{course.title}</div>
            <h2 className="mt-3 text-2xl font-bold text-slate-900">{course.title}</h2>
            <div className="mt-5">
              <p className="text-sm font-semibold uppercase tracking-[0.2em] text-slate-500">Subjects covered</p>
              <div className="mt-3 flex flex-wrap gap-2">
                {course.subjects.map((subject) => (
                  <span key={subject} className="rounded-full bg-slate-100 px-3 py-1 text-sm text-slate-700">{subject}</span>
                ))}
              </div>
            </div>
            <ul className="mt-5 space-y-3 text-slate-600">
              {course.details.map((item) => (
                <li key={item} className="flex items-center gap-3"><span className="rounded-full bg-sky-100 p-1 text-sky-700">?</span> {item}</li>
              ))}
            </ul>
          </div>
        ))}
      </div>
    </div>
  );
}

```

```

===== client/src/pages/Classes1112Page.tsx =====
const subjects = [
  {
    title: 'Physics',
    details: ['Conceptual understanding', 'Numerical problem solving', 'Class 11 / Class 12 batches', 'Doubt support', 'Regular practice'],
  },
  {
    title: 'Chemistry',
    details: ['Concept-based learning', 'Reactions and problem solving', 'Regular practice', 'Class 11 / Class 12 batches', 'Doubt support'],
  },
]

```

```

    },
    {
      title: 'Mathematics',
      details: ['Concept building', 'Problem solving', 'Regular practice', 'Class 11 / Class 12 batches', 'Faculty guidance'],
    },
  ],
];

```

```

export default function Classes1112Page() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="mb-10 text-center">
        <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Classes 11?12</p>
        <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Subject-focused senior batches</h1>
      </div>

      <div className="grid gap-6 lg:grid-cols-3">
        {subjects.map((subject) => (
          <div key={subject.title} className="card-surface p-8">
            <div className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">{subject.title}</div>
            <h2 className="mt-3 text-2xl font-bold text-slate-900">{subject.title}</h2>
            <p className="mt-3 text-sm text-slate-500">Class 11 / Class 12</p>
            <ul className="mt-5 space-y-3 text-slate-600">
              {subject.details.map((item) => (
                <li key={item} className="flex items-center gap-3"><span className="rounded-full bg-sky-100 p-1 text-sky-700">?</span> {item}</li>
              ))}
            </ul>
            <button type="button" className="primary-button mt-6 w-full">Enquire Now</button>
          </div>
        ))}
      </div>
    </div>
  );
}

```

```

===== client/src/pages/ContactPage.tsx =====
import { FormEvent, useState } from 'react';

```

```

const initialForm = {
  studentName: '',
  parentName: '',
  phone: '',
  email: '',
  className: '',
  subject: '',
  batch: '',
  message: '',
};

export default function ContactPage() {
  const [form, setForm] = useState(initialForm);
  const [errors, setErrors] = useState<Partial<Record<keyof typeof initialForm, string>>>({});
  const [status, setStatus] = useState('');

  const handleChange = (field: keyof typeof initialForm, value: string) => {
    setForm((current) => ({ ...current, [field]: value }));
    setErrors((current) => ({ ...current, [field]: '' }));
  };

  const validate = () => {
    const nextErrors: Partial<Record<keyof typeof initialForm, string>> = {};

```

```

    if (!form.studentName.trim()) nextErrors.studentName = 'Student name is required.';
    if (!form.parentName.trim()) nextErrors.parentName = 'Parent name is required.';
    if (!form.phone.trim()) nextErrors.phone = 'Phone number is required.';
    else if (!/[0-9+\-()\s]{8,}$/ .test(form.phone)) nextErrors.phone = 'Please enter a valid phone number.';
    if (!form.email.trim()) nextErrors.email = 'Email is required.';
    else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(form.email)) nextErrors.email = 'Please enter a valid email.';
    if (!form.className.trim()) nextErrors.className = 'Class is required.';
    if (!form.subject.trim()) nextErrors.subject = 'Subject is required.';
    if (!form.batch.trim()) nextErrors.batch = 'Preferred batch is required.';
    if (!form.message.trim()) nextErrors.message = 'Message is required.';
    return nextErrors;
  };

  const handleSubmit = async (event: FormEvent) => {
    event.preventDefault();
    const nextErrors = validate();
    setErrors(nextErrors);

    if (Object.keys(nextErrors).length > 0) {
      setStatus('Please correct the highlighted fields and try again.');
```

return;

```

    }

    try {
      const response = await fetch('http://localhost:4000/api/enquiries', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(form),
      });

      if (!response.ok) {
        const data = await response.json().catch(() => ({}));
        throw new Error(data.message || 'Something went wrong. Please try again.');
```

}

```

      setStatus('Thank you for contacting Manish Mishra Academy. Our team will get back to you soon.');
```

setForm(initialForm);

```

    } catch (error) {
      setStatus(error instanceof Error ? error.message : 'Something went wrong. Please try again.');
```

}

```

  };

  return (
    <div className="container-shell py-16 md:py-20">
      <div className="grid gap-10 lg:grid-cols-[0.8fr_1.2fr]">
        <div>
          <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Contact</p>
          <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Enquire with MMA</h1>
          <p className="mt-5 text-lg text-slate-600">Share your query and our team will get in touch with the right guidance for your child's academic needs.</p>
        </div>

        <form onSubmit={handleSubmit} className="card-surface p-8" noValidate>
          <div className="grid gap-5 md:grid-cols-2">
            {[
              ['studentName', 'Student Name'],
              ['parentName', 'Parent Name'],
              ['phone', 'Phone Number'],
              ['email', 'Email'],
              ['className', 'Class'],
              ['subject', 'Subject'],
              ['batch', 'Preferred Batch'],
            ]}
          </div>
        </form>
      </div>
    </div>
  );

```

```

    ].map(([field, label]) => (
      <label key={field} className="block text-sm font-medium text-slate-700">
        {label}
        <input
          value={form[field as keyof typeof initialForm]}
          onChange={(event) => handleChange(field as keyof typeof initialForm, event.target.value)}
          className="mt-2 w-full rounded-xl border border-slate-200 bg-slate-50 px-4 py-3 outline-none transition focus:border-sky-400 focus:bg-white"
          aria-invalid={Boolean(errors[field as keyof typeof initialForm])}
        />
        {errors[field as keyof typeof initialForm] && (
          <span className="mt-2 block text-xs text-red-600">{errors[field as keyof typeof initialForm]}<
        /span>
        )}
      </label>
    ))
  </div>

  <label className="mt-5 block text-sm font-medium text-slate-700">
    Message
    <textarea
      rows={5}
      value={form.message}
      onChange={(event) => handleChange('message', event.target.value)}
      className="mt-2 w-full rounded-xl border border-slate-200 bg-slate-50 px-4 py-3 outline-none transition focus:border-sky-400 focus:bg-white"
      aria-invalid={Boolean(errors.message)}
    />
    {errors.message && <span className="mt-2 block text-xs text-red-600">{errors.message}</span>}
  </label>

  {status && (
    <p className={ 'mt-6 rounded-xl px-4 py-3 text-sm ${status.includes('Thank you') ? 'bg-emerald-50 text-emerald-700' : 'bg-red-50 text-red-700'}' }>
      {status}
    </p>
  )}

  <button type="submit" className="primary-button mt-6 w-full">Submit Enquiry</button>
</form>
</div>
</div>
);
}

```

===== client/src/pages/CoursesPage.tsx =====

```

import { ArrowRight } from 'lucide-react';
import { Link } from 'react-router-dom';

const courseHighlights = [
  'Class 7',
  'Class 8',
  'Class 9',
  'Class 10',
  'Physics',
  'Chemistry',
  'Mathematics',
];

export default function CoursesPage() {
  return (
    <div className="container-shell py-16 md:py-20">

```

```

<div className="text-center">
  <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Courses</p>
  <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Academic pathways for every stage</h1>
1>
  <p className="mt-5 text-lg text-slate-600">Structured support for students from Classes 7th to 12th.</p>
</div>

<div className="mt-10 flex flex-wrap justify-center gap-3">
  {courseHighlights.map((item) => (
    <span key={item} className="rounded-full border border-sky-200 bg-sky-50 px-4 py-2 text-sm font-medium text-sky-700">{item}</span>
  ))}
</div>

<div className="mt-12 grid gap-6 lg:grid-cols-2">
  <Link to="/courses/classes-7-10" className="card-surface block p-8 transition hover:-translate-y-1 hover:shadow-xl">
    <div className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Classes 7?10</div>
    <h2 className="mt-4 text-3xl font-bold text-slate-900">Foundation years</h2>
    <p className="mt-4 text-slate-600">A complete academic base with major subject coverage, concept-building and regular assessments.</p>
    <div className="mt-6 inline-flex items-center gap-2 font-semibold text-sky-700">
      Explore class-wise courses <ArrowRight size={18} />
    </div>
  </Link>

  <Link to="/courses/classes-11-12" className="card-surface block p-8 transition hover:-translate-y-1 hover:shadow-xl">
    <div className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Classes 11?12</div>
    <h2 className="mt-4 text-3xl font-bold text-slate-900">Subject-specialised batches</h2>
    <p className="mt-4 text-slate-600">Separate batches for Physics, Chemistry and Mathematics with focused preparation and guidance.</p>
    <div className="mt-6 inline-flex items-center gap-2 font-semibold text-sky-700">
      Explore senior courses <ArrowRight size={18} />
    </div>
  </Link>
</div>
</div>
);
}

===== client/src/pages/FacilitiesPage.tsx =====
const facilities = [
  { title: 'Smart Classroom', description: 'Interactive learning spaces that make lessons easier to understand and retain.', image: 'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?auto=format&fit=crop&w=1200&q=80' },
  { title: 'AC Classroom', description: 'Comfortable, air-conditioned classrooms for focused and uninterrupted learning.', image: 'https://images.unsplash.com/photo-1503676260728-1c00da094a0b?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Library', description: 'Dedicated reading and revision space with resources for independent preparation.', image: 'https://images.unsplash.com/photo-1521587760476-6c12a4b040da?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Fibre Internet', description: 'High-speed connectivity for productive educational work during free study time.', image: 'https://images.unsplash.com/photo-1516321318423-f06f85e504b3?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Quiet Study Environment', description: 'A calm and disciplined study atmosphere that supports concentration.', image: 'https://images.unsplash.com/photo-1497633762265-9d179a990aa6?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Comfortable Seating', description: 'Ergonomic seating designed for comfortable, extended study sessions.', image: 'https://images.unsplash.com/photo-1497366754035-f200968a6e72?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Learning Resources', description: 'Subject-wise learning support materials for practice and revision.', image: 'https://images.unsplash.com/photo-1503676260728-1c00da094a0b?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Doubt Support', description: 'Students can resolve difficult concepts with faculty guidance.', image:

```

```
: 'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Regular Tests', description: 'Frequent assessments to track improvement and strengthen weak areas.',
    image: 'https://images.unsplash.com/photo-1454165804606-c3d57bc86b40?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Academic Guidance', description: 'Personal guidance and monitoring that keeps students on track.', image: 'https://images.unsplash.com/photo-1523240795612-9a054b0db644?auto=format&fit=crop&w=1200&q=80' },
];
```

```
export default function FacilitiesPage() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="text-center">
        <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Facilities</p>
        <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">A premium study environment</h1>
      </div>

      <div className="mt-12 grid gap-6 md:grid-cols-2 xl:grid-cols-3">
        {facilities.map((facility) => (
          <div key={facility.title} className="card-surface overflow-hidden">
            <img src={facility.image} alt={facility.title} className="h-52 w-full object-cover" />
            <div className="p-6">
              <h2 className="text-2xl font-bold text-slate-900">{facility.title}</h2>
              <p className="mt-3 text-slate-600">{facility.description}</p>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

```
===== client/src/pages/FaqPage.tsx =====
```

```
const faqs = [
  { question: 'Which classes are offered?', answer: 'MMA focuses on academic coaching for Classes 7th to 12th.' },
  { question: 'What subjects are available?', answer: 'For Classes 7th to 10th, all major academic subjects are covered. For Classes 11th and 12th, separate batches are offered in Physics, Chemistry and Mathematics.' },
  { question: 'What are the batch timings?', answer: 'The academy operates on batches scheduled around 4:00 PM to 7:00 PM, with one-hour learning batches.' },
  { question: 'Is library access available?', answer: 'Yes, the academy provides a dedicated study and library resource area for revision and reading.' },
  { question: 'Can students study during free time?', answer: 'Yes, students can use the available study facilities during free time for academic work.' },
  { question: 'Is doubt support available?', answer: 'Yes, faculty guidance and support are provided to help students resolve difficulties.' },
  { question: 'How can parents enquire?', answer: 'Parents and students can submit an enquiry through the contact form or reach out using the available contact details.' },
  { question: 'Are regular tests conducted?', answer: 'Yes, regular assessments are part of the learning process to track progress and identify weak areas.' },
  { question: 'Is there a student portal?', answer: 'This foundation is prepared for future student and admin portal features.' },
  { question: 'Is there a teacher portal?', answer: 'This foundation is prepared for future teacher and admin workflows.' },
  { question: 'Are fee payment policies available?', answer: 'Fee information is not specified here and should be added later from official academy records.' },
];
```

```
export default function FaqPage() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="text-center">
        <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">FAQ</p>
        <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Frequently asked questions</h1>
      </div>
    </div>
  );
}
```

```

</div>

<div className="mt-12 space-y-5">
  {faqs.map((faq) => (
    <details key={faq.question} className="card-surface p-6">
      <summary className="cursor-pointer list-none text-lg font-semibold text-slate-900">{faq.question}</s
ummary>
      <p className="mt-4 text-slate-600">{faq.answer}</p>
    </details>
  ))}
</div>
</div>
);
}

```

```

===== client/src/pages/FeesPage.tsx =====
import { useEffect, useState } from 'react';

```

```

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000';
type Fee = { id: string; totalAmountPaise: number; paidAmountPaise: number; summary: { outstandingAmountPaise: n
umber; status: string }; feeStructure: { name: string; description?: string }; payments: Array<{ id: string; amo
untPaise: number; status: string; receipt?: { id: string; receiptNumber: string } | null }> };
const rupees = (paise: number) => `₹${(paise / 100).toLocaleString('en-IN', { minimumFractionDigits: 2 })}`;

export default function FeesPage({ mode }: { mode: 'student' | 'admin' }) {
  const [fees, setFees] = useState<Fee[]>([]);
  const [payments, setPayments] = useState<Array<Record<string, any>>>([]);
  const [students, setStudents] = useState<Array<{ id: string; user: { fullName: string; email: string } }>>([])
;
  const [structures, setStructures] = useState<Array<{ id: string; name: string; className: string }>>([]);
  const [assignment, setAssignment] = useState({ studentId: '', feeStructureId: '' });
  const [amounts, setAmounts] = useState<Record<string, string>>({});
  const [message, setMessage] = useState('');
  const [error, setError] = useState('');
  const [structure, setStructure] = useState({ name: '', description: '', className: '', amount: '' });

  async function load() {
    const endpoint = mode === 'student' ? '/api/student/fees' : '/api/admin/fees';
    const response = await fetch(`${API_URL}${endpoint}`, { credentials: 'include' });
    const payload = await response.json();
    if (!response.ok) throw new Error(payload.message || 'Unable to load fee records.');
```

```

    setFees(payload.data || []);
    if (mode === 'admin') { const [paymentResponse, studentResponse, structureResponse] = await Promise.all([fet
ch(`${API_URL}/api/admin/payments`, { credentials: 'include' }), fetch(`${API_URL}/api/admin/students`, { creden
tials: 'include' }), fetch(`${API_URL}/api/admin/fee-structures`, { credentials: 'include' })]); const paymentPa
yload = await paymentResponse.json(); const studentPayload = await studentResponse.json(); const structurePayloa
d = await structureResponse.json(); if (paymentResponse.ok) setPayments(paymentPayload.data || []); if (studentR
esponse.ok) setStudents(studentPayload.data || []); if (structureResponse.ok) setStructures(structurePayload.dat
a || []); }
  }

  useEffect(() => { load().catch((loadError: unknown) => setError(loadError instanceof Error ? loadError.message
: 'Unable to load fee records.')); }, [mode]);

  async function createOrder(fee: Fee) {
    const amountPaise = Math.round(Number(amounts[fee.id]) * 100);
    if (!amountPaise || amountPaise > fee.summary.outstandingAmountPaise) { setError('Enter a valid amount withi
n the outstanding balance.');
```

```

    return; }
    const idempotencyKey = `mma-${fee.id}-${amountPaise}`;
    const response = await fetch(`${API_URL}/api/student/payments/order`, { method: 'POST', credentials: 'includ
e', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ studentFeeId: fee.id, amountPaise, i
dempotencyKey }) });
    const payload = await response.json();
  }
}

```

```

    if (!response.ok) { setError(payload.message || 'Unable to create payment.');
```

```

    if (!payload.data.keyId) { setMessage('Sandbox order created. Configure Razorpay test credentials to open the gateway.');
```

```

    const razorpayWindow = window as Window & { Razorpay?: new (options: Record<string, unknown>) => { open: () => void } };
```

```

    if (!razorpayWindow.Razorpay) { const script = document.createElement('script'); script.src = 'https://checkout.razorpay.com/v1/checkout.js'; script.onload = () => openGateway(payload.data); document.body.appendChild(script); } else openGateway(payload.data);
```

```

    function openGateway(order: { orderId: string; amountPaise: number; keyId: string }) {
        const Razorpay = (window as unknown as { Razorpay: new (options: Record<string, unknown>) => { open: () => void } }).Razorpay;
```

```

        const checkout = new Razorpay({ key: order.keyId, amount: order.amountPaise, currency: 'INR', name: 'Manish Mishra Academy', description: fee.feeStructure.name, order_id: order.orderId, handler: async (result: { razorpay_order_id: string; razorpay_payment_id: string; razorpay_signature: string }) => { const verifyResponse = await fetch(`${API_URL}/api/student/payments/verify`, { method: 'POST', credentials: 'include', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ paymentRecordId: payload.data.paymentRecordId, razorpayOrderId: result.razorpay_order_id, razorpayPaymentId: result.razorpay_payment_id, razorpaySignature: result.razorpay_signature }) }); setMessage(verifyResponse.ok ? 'Payment verified successfully.' : 'Payment could not be completed. Please try again.');
```

```

        if (verifyResponse.ok) load(); }, modal: { ondismiss: () => setMessage('Payment could not be completed. Please try again.') });
        checkout.open();
    }
}

async function createStructure() {
    const response = await fetch(`${API_URL}/api/admin/fee-structures`, { method: 'POST', credentials: 'include', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ name: structure.name, description: structure.description, className: structure.className, totalAmountPaise: Math.round(Number(structure.amount) * 100) });
    const payload = await response.json();
    if (!response.ok) { setError(payload.message || 'Unable to create fee structure.');
```

```

    return; }
    setMessage('Fee structure configured. Assign it to students through the authorized administration workflow.');
```

```

    setStructure({ name: '', description: '', className: '', amount: '' });
}

async function assignFee() {
    const response = await fetch(`${API_URL}/api/admin/student-fees`, { method: 'POST', credentials: 'include', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(assignment) });
    const payload = await response.json();
    if (!response.ok) { setError(payload.message || 'Unable to assign fee.');
```

```

    return; }
    setMessage('Fee assigned to the student.');
```

```

    setAssignment({ studentId: '', feeStructureId: '' }); load();
}

return <div className="container-shell py-12"><p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">{mode === 'student' ? 'Student corner' : 'Administration'}</p><h1 className="mt-3 text-4xl font-bold text-slate-900">{mode === 'student' ? 'Fees & payments' : 'Fee management'}</h1><p className="mt-3 text-slate-600">{mode === 'student' ? 'View your assigned fees, balance, payment history, and receipts.' : 'Review balances and verified gateway transactions. Successful payments are not manually editable.'}</p>{message && <p className="mt-6 rounded-lg bg-emerald-50 px-4 py-3 text-emerald-700">{message}</p>}{error && <p className="mt-6 rounded-lg bg-red-50 px-4 py-3 text-red-700">{error}</p>}
```

```

    {mode === 'student' && !fees.length && <div className="card-surface mt-8 p-8"><h2 className="text-xl font-bold text-slate-900">No fee assigned yet</h2><p className="mt-2 text-slate-600">Your fee structure has not been configured by an authorized academy administrator.</p></div>}
```

```

    {mode === 'admin' && <div className="card-surface mt-8 grid gap-4 p-6 md:grid-cols-2"><h2 className="md:col-span-2 text-xl font-bold text-slate-900">Configure fee structure</h2>{([ 'name', 'description', 'className', 'amount' ] as const).map((field) => <label className="text-sm font-medium text-slate-700" key={field}>{field === 'amount' ? 'Total amount in INR' : field[0].toUpperCase() + field.slice(1)}<input required={field !== 'description'} type={field === 'amount' ? 'number' : 'text'} min={field === 'amount' ? '0.01' : undefined} step={field === 'amount' ? '0.01' : undefined} value={structure[field]} onChange={(event) => setStructure({ ...structure, [field]: event.target.value })} className="mt-2 w-full rounded-xl border border-slate-200 px-4 py-3" /></label>)}<button type="button" onClick={createStructure} className="primary-button md:col-span-2">Save fee structure</div>}</div>
```

```

===== client/src/pages/HomePage.tsx =====
import { ArrowRight, BookOpen, Building2, Clock3, GraduationCap, Library, Wifi, ShieldCheck } from 'lucide-react';
import { Link } from 'react-router-dom';

const highlights = [
  'Classes 7?12',
  'Smart Classrooms',
  'AC Rooms',
  'Library',
  'Fibre Internet',
  'Personal Guidance',
  'Quiet Study Environment',
];

const stats = [
  { label: 'Classes', value: '7?12' },
  { label: 'Focus', value: 'Personal' },
  { label: 'Learning', value: 'Concept-first' },
  { label: 'Support', value: 'Doubt Solving' },
];

export default function HomePage() {
  return (
    <>
      <section className="container-shell grid items-center gap-10 pb-16 pt-16 md:pt-20 lg:grid-cols-[1.15fr_0.8

```

```

5fr]">
  <div>
    <span className="inline-flex rounded-full border border-sky-200 bg-sky-50 px-4 py-2 text-xs font-semib
old uppercase tracking-[0.2em] text-sky-700">
      Premium academic learning
    </span>
    <h1 className="mt-6 text-4xl font-black tracking-tight text-slate-900 sm:text-5xl lg:text-6xl">
      MANISH MISHRA ACADEMY
    </h1>
    <p className="mt-5 text-2xl font-semibold text-sky-700">Learn Better. Think Smarter. Achieve More.</p>
    <p className="mt-6 max-w-xl text-lg text-slate-600">
      Focused academic coaching for Classes 7th to 12th, with personal guidance, modern classrooms, dedica
ted study facilities and a quiet environment designed to help students learn with confidence.
    </p>
    <div className="mt-8 flex flex-wrap gap-4">
      <Link to="/courses" className="primary-button gap-2">
        Explore Classes <ArrowRight size={18} />
      </Link>
      <Link to="/contact" className="secondary-button">
        Enquire Now
      </Link>
      <Link to="/timetable" className="secondary-button">
        View Timetable
      </Link>
    </div>

    <div className="mt-8 flex items-center gap-3 text-sm text-slate-700">
      <span className="font-semibold text-slate-900">Student & Teacher Login</span>
      <Link to="/login" className="inline-flex items-center gap-1 text-sky-700 hover:underline">
        Access portal <ArrowRight size={14} />
      </Link>
    </div>

    <div className="mt-10 grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
      {stats.map((stat) => (
        <div key={stat.label} className="card-surface p-4">
          <div className="text-xl font-bold text-slate-900">{stat.value}</div>
          <div className="mt-1 text-sm text-slate-600">{stat.label}</div>
        </div>
      ))}
    </div>

    <div className="relative">
      <div className="card-surface overflow-hidden p-4 shadow-[0_20px_50px_rgba(14,116,144,0.14)]">
        
      </div>
      <div className="absolute -bottom-5 left-5 rounded-2xl border border-sky-100 bg-white p-4 shadow-xl sha
dow-sky-100">
        <div className="flex items-center gap-3">
          <div className="rounded-xl bg-emerald-100 p-2 text-emerald-700"><ShieldCheck size={22} /></div>
          <div>
            <div className="text-sm text-slate-500">Academic support</div>
            <div className="font-semibold text-slate-900">Guided learning</div>
          </div>
        </div>
      </div>
    </div>

```

```

</section>

<section className="border-y border-slate-200 bg-white/70">
  <div className="container-shell flex flex-wrap justify-center gap-4 py-6 text-center text-sm font-medium text-slate-700">
    {highlights.map((item) => (
      <span key={item} className="rounded-full border border-slate-200 bg-slate-50 px-4 py-2">
        {item}
      </span>
    ))}
  </div>
</section>

<section className="container-shell py-20">
  <div className="text-center">
    <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Why MMA</p>
    <h2 className="section-title mt-3">A structured learning environment that helps students grow</h2>
  </div>

  <div className="mt-12 grid gap-6 md:grid-cols-2 xl:grid-cols-4">
    {[
      { icon: BookOpen, title: 'Concept-based learning', text: 'Students learn the fundamentals clearly rather than memorising without understanding.' },
      { icon: GraduationCap, title: 'Personal attention', text: 'Each learner gets guidance, support and academic monitoring in a focused setting.' },
      { icon: Building2, title: 'Smart classrooms', text: 'Modern teaching methods make lessons more interactive and easier to follow.' },
      { icon: Wifi, title: 'Fibre internet', text: 'Students can make productive use of free study time with online learning support.' },
      { icon: Library, title: 'Library access', text: 'A quiet, resourceful space for revision, reading and independent study.' },
      { icon: Clock3, title: 'Regular assessments', text: 'Frequent tests help identify learning gaps early and improve performance steadily.' },
      { icon: ShieldCheck, title: 'Disciplined environment', text: 'Punctuality, focus and consistency strengthen good study habits.' },
      { icon: ArrowRight, title: 'Doubt support', text: 'Students can ask questions and get timely guidance without confusion lingering for long.' },
    ]}.map(({ icon: Icon, title, text }) => (
      <div key={title} className="card-surface p-6 transition hover:-translate-y-1 hover:shadow-xl">
        <div className="mb-4 inline-flex rounded-xl bg-sky-100 p-3 text-sky-700"><Icon size={22} /></div>
        <h3 className="text-xl font-bold text-slate-900">{title}</h3>
        <p className="mt-3 text-sm leading-6 text-slate-600">{text}</p>
      </div>
    ))}
  </div>
</section>

<section className="bg-slate-950 py-20 text-white">
  <div className="container-shell grid gap-10 lg:grid-cols-[0.9fr_1.1fr]">
    <div>
      <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-300">About MMA</p>
      <h2 className="mt-3 text-3xl font-bold md:text-4xl">A learning culture built around clarity, discipline and progress</h2>
      <p className="mt-5 text-slate-300">
        Manish Mishra Academy focuses on building strong academic fundamentals. Students are encouraged to understand concepts clearly, practice regularly, solve doubts confidently and improve with consistency.
      </p>
      <ul className="mt-6 space-y-4 text-slate-200">
        {[ 'Conceptual learning', 'Regular practice', 'Doubt resolution', 'Academic monitoring', 'Student discipline', 'Consistent academic progress' ].map((item) => (
          <li key={item} className="flex items-center gap-3"><span className="rounded-full bg-sky-500/20 p-1 text-sky-300">?</span> {item}</li>
        ))}
      </ul>
    </div>
  </div>
</section>

```

```

    ))}
  </ul>
</div>
<div className="grid gap-6 sm:grid-cols-2">
  {[
    { title: '01 ? Learn', text: 'Understand the concept clearly in class.' },
    { title: '02 ? Practice', text: 'Solve questions and assignments daily.' },
    { title: '03 ? Ask', text: 'Clear doubts with faculty guidance.' },
    { title: '04 ? Test', text: 'Track strengths and weak areas regularly.' },
    { title: '05 ? Analyse', text: 'Review performance to identify improvements.' },
    { title: '06 ? Improve', text: 'Repeat and strengthen the learning cycle.' },
  ]}.map((step) => (
    <div key={step.title} className="rounded-2xl border border-slate-800 bg-slate-900 p-6">
      <div className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-300">{step.title.split
('')[0].trim()}</div>
      <h3 className="mt-3 text-xl font-bold text-white">{step.title}</h3>
      <p className="mt-3 text-sm leading-6 text-slate-300">{step.text}</p>
    </div>
  ))}
</div>
</div>
</section>
</>
);
}

```

```

===== client/src/pages/LoginPage.tsx =====
import { FormEvent, useState } from 'react';
import { useNavigate } from 'react-router-dom';

type LoginResponse = {
  message?: string;
  redirectTo?: string;
};

export default function LoginPage() {
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError('');
    setIsSubmitting(true);

    try {
      const response = await fetch(`${import.meta.env.VITE_API_URL || 'http://localhost:4000'}/api/auth/login`,
{
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
      body: JSON.stringify({ email, password }),
    });
      const data = (await response.json()) as LoginResponse;

      if (!response.ok || !data.redirectTo) {
        throw new Error(data.message || 'Unable to sign in. Please check your details.');

```

```

    } catch (loginError) {
      setError(loginError instanceof Error ? loginError.message : 'Unable to sign in.');
```

```

    } finally {
      setIsSubmitting(false);
    }
  }

  return (
    <div className="container-shell flex items-center justify-center py-20">
      <div className="card-surface w-full max-w-md p-8">
        <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">MMA portal access</p>
        <h1 className="mt-4 text-3xl font-bold text-slate-900">Login</h1>
        <p className="mt-3 text-slate-600">Sign in with the account issued by Manish Mishra Academy.</p>
        <form className="mt-6 space-y-4" onSubmit={handleSubmit}>
          <label className="block text-sm font-medium text-slate-700">
            Email
            <input value={email} onChange={(event) => setEmail(event.target.value)} required type="email" className="mt-2 w-full rounded-xl border border-slate-200 bg-slate-50 px-4 py-3" placeholder="your.email@example.com" />
          </label>
          <label className="block text-sm font-medium text-slate-700">
            Password
            <input value={password} onChange={(event) => setPassword(event.target.value)} required type="password" className="mt-2 w-full rounded-xl border border-slate-200 bg-slate-50 px-4 py-3" placeholder="Enter password" />
          </label>
          {error && <p role="alert" className="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-700">{error}</p>}
          <button disabled={!isSubmitting} type="submit" className="primary-button w-full disabled:cursor-not-allowed disabled:opacity-60">
            {isSubmitting ? 'Signing in...' : 'Login'}
          </button>
        </form>
      </div>
    </div>
  );
}

```

```

===== client/src/pages/NotFoundPage.tsx =====

```

```

import { Link } from 'react-router-dom';

```

```

export default function NotFoundPage() {
  return (
    <div className="container-shell flex min-h-[60vh] flex-col items-center justify-center text-center py-16">
      <div className="text-6xl font-black text-sky-600">404</div>
      <h1 className="mt-4 text-3xl font-bold text-slate-900">Page not found</h1>
      <p className="mt-3 max-w-lg text-slate-600">The page you are looking for does not exist or may have moved.
    </p>
    <Link to="/" className="primary-button mt-8">Go to homepage</Link>
  </div>
  );
}

```

```

===== client/src/pages/PortalPage.tsx =====

```

```

import { useEffect, useState } from 'react';
import { useNavigate, useLocation } from 'react-router-dom';

```

```

type PortalData = {
  user?: { fullName?: string; role?: string };
  students?: number;
  teachers?: number;
  enquiries?: number;
  courses?: number;
}

```

```

resources?: number;
timetable?: Array<{ day: string; subject?: string; startTime: string; endTime: string }>;
announcements?: Array<{ title: string; message: string }>;
tests?: Array<{ title: string; date?: string; status: string }>;
assignments?: Array<{ className: string; subject: string; batch?: string }>;
notifications?: Array<{ title: string; message: string }>;
enrollments?: Array<{ className: string; subject: string; batch?: string }>;
assignedTeachers?: Array<{ className: string; subject: string; teacher?: { user?: { fullName?: string; email?: string } } }>;
};

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000';

export default function PortalPage() {
  const navigate = useNavigate();
  const location = useLocation();
  const [data, setData] = useState<PortalData>({});
  const [error, setError] = useState('');

  const isAdmin = location.pathname.startsWith('/admin');
  const roleLabel = isAdmin ? 'Administration portal' : location.pathname.startsWith('/teacher') ? 'Teacher portal' : 'Student portal';
  const endpoint = isAdmin ? '/api/admin/dashboard' : location.pathname.startsWith('/teacher') ? '/api/teacher/dashboard' : '/api/student/dashboard';

  useEffect(() => {
    fetch(`${API_URL}${endpoint}`, { credentials: 'include' })
      .then(async (response) => {
        const payload = await response.json();
        if (!response.ok) throw new Error(payload.message || 'Unable to load this portal.');
```

setData(payload.data || {});

```

      })
      .catch((portalError: unknown) => {
        setError(portalError instanceof Error ? portalError.message : 'Unable to load this portal.');
```

});

```

  }, [endpoint]);

  async function logout() {
    await fetch(`${API_URL}/api/auth/logout`, { method: 'POST', credentials: 'include' });
    navigate('/login');
  }

  const stats = [
    ['Students', data.students],
    ['Teachers', data.teachers],
    ['Enquiries', data.enquiries],
    ['Courses', data.courses],
  ].filter(([, value]) => value !== undefined);

  return (
    <div className="container-shell py-12">
      <div className="flex flex-wrap items-end justify-between gap-4">
        <div>
          <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">{roleLabel}</p>
          <h1 className="mt-3 text-4xl font-bold text-slate-900">Welcome back{data.user?.fullName ? ', ${data.user.fullName}' : ''}</h1>
          <p className="mt-3 text-slate-600">Your MMA workspace is ready with access controlled by your account role.</p>
        </div>
        <button type="button" onClick={logout} className="secondary-button">Sign out</button>
      </div>
    </div>
  );
}

```

```

<div className="mt-8 flex flex-wrap gap-3">
  {location.pathname.startsWith('/student')} && <><a className="secondary-button" href="/student/resources"
>Study material</a><a className="secondary-button" href="/student/tests">Tests</a><a className="secondary-button
" href="/student/timetable">Timetable</a><a className="secondary-button" href="/student/fees">Fees</a></>
  {location.pathname.startsWith('/teacher')} && <><a className="secondary-button" href="/teacher/resources"
>My resources</a><a className="secondary-button" href="/teacher/tests">Tests</a><a className="secondary-button"
href="/teacher/timetable">Timetable</a></>
  {isAdmin && <><a className="secondary-button" href="/admin/tests">All tests</a><a className="secondary-b
utton" href="/admin/fees">Fees</a><a className="secondary-button" href="/admin/payments">Payments</a></>
</div>

{error ? (
  <div className="card-surface mt-8 p-6 text-red-700">{error}. Please sign in again.</div>
) : (
  <>
    {stats.length > 0 && (
      <div className="mt-10 grid gap-4 sm:grid-cols-2 lg:grid-cols-4">
        {stats.map(([label, value]) => (
          <div className="card-surface p-6" key={label as string}>
            <p className="text-sm text-slate-500">{label}</p>
            <p className="mt-2 text-3xl font-bold text-slate-900">{value}</p>
          </div>
        ))}
      </div>
    )}
  </div>
)}

{data.assignments && data.assignments.length > 0 && <section className="mt-10"><h2 className="section-
title text-2xl">Classes I Teach</h2><div className="mt-5 grid gap-4 md:grid-cols-3">{data.assignments.map((item)
=> <div className="card-surface p-5" key={` ${item.className} - ${item.subject} `}><p className="font-semibold text
-slate-900">{item.className}</p><p className="mt-2 text-sm text-slate-600">{item.subject}{item.batch ? ' ? ${ite
m.batch}' : ''}</p></div>)}</div></section>}

{data.enrollments && data.enrollments.length > 0 && <section className="mt-10"><h2 className="section-
title text-2xl">My Classes & Subjects</h2><div className="mt-5 grid gap-4 md:grid-cols-3">{data.enrollments.map(
(item) => <div className="card-surface p-5" key={` ${item.className} - ${item.subject} `}><p className="font-semibol
d text-slate-900">{item.className}</p><p className="mt-2 text-sm text-slate-600">{item.subject}{item.batch ? ' ?
${item.batch}' : ''}</p></div>)}</div></section>}

{data.assignedTeachers && data.assignedTeachers.length > 0 && <section className="mt-10"><h2 className
="section-title text-2xl">My Teachers</h2><div className="mt-5 grid gap-4 md:grid-cols-3">{data.assignedTeachers
.map((item) => <div className="card-surface p-5" key={` ${item.subject} - ${item.teacher?.user?.email} `}><p classNa
me="font-semibold text-slate-900">{item.subject}</p><p className="mt-2 text-sm text-slate-600">{item.teacher?.us
er?.fullName || 'Assigned faculty'}</p></div>)}</div></section>}

{data.timetable && data.timetable.length > 0 && (
  <section className="mt-10">
    <h2 className="section-title text-2xl">Upcoming timetable</h2>
    <div className="mt-5 grid gap-4 md:grid-cols-2">
      {data.timetable.map((item, index) => (
        <div className="card-surface p-5" key={` ${item.day} - ${index} `}>
          <p className="font-semibold text-slate-900">{item.day} ? {item.subject || 'Class session'}</
p>
          <p className="mt-2 text-sm text-slate-600">{item.startTime} - {item.endTime}</p>
        </div>
      ))}
    </div>
  </section>
)}

{data.announcements && data.announcements.length > 0 && (
  <section className="mt-10">
    <h2 className="section-title text-2xl">Announcements</h2>

```

```

        <div className="mt-5 space-y-4">
          {data.announcements.map((item) => <div className="card-surface p-5" key={item.title}><h3 classNa
me="font-semibold text-slate-900">{item.title}</h3><p className="mt-2 text-sm text-slate-600">{item.message}</p>
</div>)}
        </div>
      </section>
    )}

    {data.tests && data.tests.length > 0 && (
      <section className="mt-10">
        <h2 className="section-title text-2xl">Tests</h2>
        <div className="mt-5 grid gap-4 md:grid-cols-3">
          {data.tests.map((item) => <div className="card-surface p-5" key={item.title}><h3 className="font
-semibold text-slate-900">{item.title}</h3><p className="mt-2 text-sm text-slate-600">{item.date || 'Date to be
announced'} ? {item.status}</p></div>)}
        </div>
      </section>
    )}
  </>
) }
</div>
);
}
===== client/src/pages/ResourcesPage.tsx =====
import { FormEvent, useEffect, useState } from 'react';

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000';
const resourceTypes = ['Notes', 'PDF', 'Assignment', 'Worksheet', 'Question paper', 'Revision material', 'Refere
nce material'];

type Resource = {
  id: string;
  title: string;
  description?: string;
  className: string;
  subject: string;
  topic: string;
  type: string;
  fileName: string;
  createdAt: string;
  teacher?: { user?: { fullName?: string } };
};

type Assignment = { className: string; subject: string; batch?: string };

export default function ResourcesPage({ mode }: { mode: 'student' | 'teacher' }) {
  const [resources, setResources] = useState<Resource[]>([]);
  const [assignments, setAssignments] = useState<Assignment[]>([]);
  const [search, setSearch] = useState('');
  const [filters, setFilters] = useState({ className: '', subject: '', type: '' });
  const [form, setForm] = useState({ title: '', description: '', className: '', subject: '', topic: '', type: 'N
otes', file: null as File | null });
  const [message, setMessage] = useState('');
  const [error, setError] = useState('');

  async function loadResources() {
    const query = new URLSearchParams({ search, ...filters });
    const response = await fetch(`${API_URL}/api/${mode}/resources?${query}`, { credentials: 'include' });
    const payload = await response.json();
    if (!response.ok) throw new Error(payload.message || 'Unable to load resources.');
```

```

    }

    useEffect(() => { loadResources().catch((loadError: unknown) => setError(loadError instanceof Error ? loadError.message : 'Unable to load resources.')); }, [mode, search, filters]);

    async function download(resource: Resource) {
        const response = await fetch(`${API_URL}/api/resources/${resource.id}/download`, { credentials: 'include' });
        if (!response.ok) { setError('You are not authorized to download this file.');
```

return;

```
        const blob = await response.blob();
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a'); link.href = url; link.download = resource.fileName; link.click();
        URL.revokeObjectURL(url);
    }

    async function publish(event: FormEvent<HTMLFormElement>) {
        event.preventDefault(); setError(''); setMessage('');
        if (!form.file) { setError('Choose a PDF, image, or text file first.');
```

return;

```
        const fileData = await new Promise<string>((resolve, reject) => { const reader = new FileReader(); reader.onload = () => resolve(String(reader.result)); reader.onerror = reject; reader.readAsDataURL(form.file as File); });
        const response = await fetch(`${API_URL}/api/teacher/resources`, { method: 'POST', credentials: 'include', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ ...form, file: undefined, fileName: form.file.name, mimeType: form.file.type, fileData }) });
        const payload = await response.json();
        if (!response.ok) { setError(payload.message || 'Unable to publish resource.');
```

return;

```
        setMessage('Resource published successfully.');
```

setForm({ title: '', description: '', className: '', subject: '', topic: '', type: 'Notes', file: null }); loadResources();

}

```
    return (
        <div className="container-shell py-12">
            <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">{mode === 'student' ? 'Student corner' : 'Teacher workspace'}</p>
            <h1 className="mt-3 text-4xl font-bold text-slate-900">{mode === 'student' ? 'Study resources' : 'My resource library'}</h1>
            <p className="mt-3 max-w-2xl text-slate-600">{mode === 'student' ? 'Find notes, assignments, worksheets, and revision material available for your enrolled classes.' : 'Publish academic material only to classes and subjects assigned to your teacher profile.'}</p>

            {mode === 'teacher' && <form onSubmit={publish} className="card-surface mt-8 grid gap-4 p-6 md:grid-cols-2">
                <h2 className="md:col-span-2 text-xl font-bold text-slate-900">Publish a resource</h2>
                {[('title', 'description', 'topic') as const].map((field) => <label key={field} className="text-sm font-medium text-slate-700">{field[0].toUpperCase() + field.slice(1)}<input required={field !== 'description'} value={form[field]} onChange={(event) => setForm({ ...form, [field]: event.target.value })} className="mt-2 w-full rounded-xl border border-slate-200 px-4 py-3" /></label>)}
                <label className="text-sm font-medium text-slate-700">Class<select required value={form.className} onChange={(event) => setForm({ ...form, className: event.target.value })} className="mt-2 w-full rounded-xl border border-slate-200 px-4 py-3"><option value="">Choose assigned class</option>{[...new Set(assignments.map((item) => item.className))].map((item) => <option key={item}>{item}</option>)}</select></label>
                <label className="text-sm font-medium text-slate-700">Subject<select required value={form.subject} onChange={(event) => setForm({ ...form, subject: event.target.value })} className="mt-2 w-full rounded-xl border border-slate-200 px-4 py-3"><option value="">Choose assigned subject</option>{[...new Set(assignments.filter((item) => !form.className || item.className === form.className).map((item) => item.subject))].map((item) => <option key={item}>{item}</option>)}</select></label>
                <label className="text-sm font-medium text-slate-700">Type<select value={form.type} onChange={(event) => setForm({ ...form, type: event.target.value })} className="mt-2 w-full rounded-xl border border-slate-200 px-4 py-3">{resourceTypes.map((item) => <option key={item}>{item}</option>)}</select></label>
                <label className="text-sm font-medium text-slate-700">File<input required type="file" accept=".pdf,.jpg,.jpeg,.png,.txt" onChange={(event) => setForm({ ...form, file: event.target.files?.[0] || null })} className="mt-2 block w-full text-sm" /></label>
            </form>
        </div>
    )

```

```

    <button className="primary-button md:col-span-2" type="submit">Publish resource</button>
  </form>

  <div className="card-surface mt-8 grid gap-4 p-5 md:grid-cols-4">
    <input value={search} onChange={(event) => setSearch(event.target.value)} placeholder="Search resources"
    className="rounded-xl border border-slate-200 px-4 py-3 md:col-span-2" />
    {[('className', 'subject', 'type') as const].map((filter) => <input key={filter} value={filters[filter]}
    onChange={(event) => setFilters({ ...filters, [filter]: event.target.value })} placeholder={filter === 'className' ? 'Class' : filter[0].toUpperCase() + filter.slice(1)} className="rounded-xl border border-slate-200 px-4 py-3" />)}
  </div>

  {message && <p className="mt-5 rounded-lg bg-emerald-50 px-4 py-3 text-emerald-700">{message}</p>}
  {error && <p className="mt-5 rounded-lg bg-red-50 px-4 py-3 text-red-700">{error}</p>}
  <div className="mt-8 grid gap-5 md:grid-cols-2">
    {resources.map((resource) => <article className="card-surface p-6" key={resource.id}><div className="flex
    x items-start justify-between gap-4"><div><p className="text-xs font-semibold uppercase tracking-wider text-sky-700">{resource.type}</p><h2 className="mt-2 text-xl font-bold text-slate-900">{resource.title}</h2></div><button
    type="button" onClick={() => download(resource)} className="secondary-button px-4 py-2">Download</button></div>
    <p className="mt-3 text-sm text-slate-600">{resource.description || resource.topic}</p><p className="mt-4 text-sm text-slate-500">{resource.className} ? {resource.subject} ? {resource.teacher?.user?.fullName || 'MMA faculty'}</p><p className="mt-2 text-xs text-slate-400">Uploaded {new Date(resource.createdAt).toLocaleDateString()}</p>
    </article>)}
  </div>
  {!resources.length && !error && <p className="mt-8 text-slate-600">No resources match your current filters
  </p>}
  </div>
  );
}

===== client/src/pages/StudyEnvironmentPage.tsx =====
const experience = [
  'Attend classes and stay consistent',
  'Revise topics with guided support',
  'Read books and study resources',
  'Use educational internet resources',
  'Ask doubts and receive timely guidance',
  'Prepare for tests in a calm environment',
  'Study during free time productively',
  'Receive academic guidance and follow-up',
];

export default function StudyEnvironmentPage() {
  return (
    <div className="container-shell py-16 md:py-20">
      <div className="grid gap-10 lg:grid-cols-[0.9fr_1.1fr] lg:items-center">
        <div>
          <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Study environment</p>
          <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">More than just a classroom</h1>
          <p className="mt-5 text-lg text-slate-600">
            Students can attend classes, revise concepts, read books, use educational internet resources, ask do
            ubts and study during free time in a focused environment built for progress.
          </p>
        </div>
        <div className="card-surface overflow-hidden p-3">
          
        </div>
      </div>

      <div className="mt-12 grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {experience.map((item) => (

```

```

        <div key={item} className="card-surface p-5 text-slate-800">{item}</div>
    )}}
</div>
</div>
);
}

===== client/src/pages/TimetablePage.tsx =====
const times = ['4:00 PM ? 5:00 PM', '5:00 PM ? 6:00 PM', '6:00 PM ? 7:00 PM'];

export default function TimetablePage() {
    return (
        <div className="container-shell py-16 md:py-20">
            <div className="text-center">
                <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">Timetable</p>
                <h1 className="mt-4 text-4xl font-black text-slate-900 md:text-5xl">Daily batch schedule</h1>
            </div>

            <div className="mt-10 overflow-hidden rounded-3xl border border-slate-200 bg-white shadow-lg">
                <div className="grid grid-cols-4 bg-slate-900 text-white">
                    <div className="px-6 py-4 font-semibold">Time</div>
                    <div className="px-6 py-4 font-semibold">Batch</div>
                    <div className="px-6 py-4 font-semibold">Class</div>
                    <div className="px-6 py-4 font-semibold">Room</div>
                </div>
                {times.map((time) => (
                    <div key={time} className="grid grid-cols-4 border-t border-slate-200 text-slate-700">
                        <div className="px-6 py-5 font-medium">{time}</div>
                        <div className="px-6 py-5">Batch</div>
                        <div className="px-6 py-5">Editable</div>
                        <div className="px-6 py-5">To be assigned</div>
                    </div>
                ))}
            </div>

            <div className="mt-10 rounded-2xl border border-dashed border-sky-200 bg-sky-50 p-6 text-slate-700">
                This timetable is intentionally designed for easy database updates later. Administrators can assign clas
s, subject, teacher, day, batch, room and time slots as needed.
            </div>
        </div>
    );
}

===== server/package.json =====
{
    "name": "mma-server",
    "private": true,
    "version": "1.0.0",
    "type": "module",
    "main": "dist/server.js",
    "scripts": {
        "dev": "tsx watch src/server.ts",
        "build": "tsc -p tsconfig.json",
        "start": "node dist/server.js",
        "db:generate": "prisma generate",
        "db:push": "prisma db push",
        "db:seed": "tsx prisma/seed.ts"
    },
    "dependencies": {
        "@prisma/client": "^5.22.0",
        "bcryptjs": "^2.4.3",
        "cors": "^2.8.5",

```

```

    "dotenv": "^16.4.5",
    "express": "^4.21.1",
    "jsonwebtoken": "^9.0.2",
    "prisma": "^5.22.0",
    "zod": "^3.23.1"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.1",
    "@types/cors": "^2.8.5",
    "@types/express": "^5.0.0",
    "@types/jsonwebtoken": "^9.0.5",
    "@types/node": "^22.10.1",
    "tsx": "^4.19.2",
    "typescript": "^5.7.2"
  }
}

==== server/src/config.ts ====
import dotenv from 'dotenv';

dotenv.config({ path: '../.env' });

export const config = {
  port: Number(process.env.PORT || 4000),
  jwtSecret: process.env.JWT_SECRET || 'development_secret',
  clientUrl: process.env.CLIENT_URL || 'http://localhost:5173',
  adminEmail: process.env.ADMIN_EMAIL || 'admin@mmacademy.com',
  adminPassword: process.env.ADMIN_PASSWORD || 'Mma@2026',
  paymentKeyId: process.env.PAYMENT_KEY_ID || '',
  paymentKeySecret: process.env.PAYMENT_KEY_SECRET || '',
  paymentWebhookSecret: process.env.PAYMENT_WEBHOOK_SECRET || '',
  paymentSandbox: process.env.PAYMENT_SANDBOX !== 'false',
};

==== server/src/lib/auth.ts ====
import jwt from 'jsonwebtoken';
import { config } from '../config.js';

export type AuthPayload = {
  userId: string;
  email: string;
  role: string;
};

export function signToken(payload: AuthPayload) {
  return jwt.sign(payload, config.jwtSecret, { expiresIn: '7d' });
}

export function verifyToken(token: string) {
  return jwt.verify(token, config.jwtSecret) as AuthPayload;
}

==== server/src/lib/prisma.ts ====
import { PrismaClient } from '@prisma/client';

export const prisma = new PrismaClient();

==== server/src/server.ts ====
import cors from 'cors';
import express from 'express';
import bcrypt from 'bcryptjs';
import { createHmac, randomUUID } from 'node:crypto';

```

```

import { mkdir, writeFile } from 'node:fs/promises';
import path from 'node:path';
import { z } from 'zod';
import { config } from './config.js';
import { signToken, verifyToken } from './lib/auth.js';
import { prisma } from './lib/prisma.js';

const app = express();
const loginAttempts = new Map<string, { count: number; firstAttempt: number }>();
const VALID_ROLES = ['HEAD', 'SUPERVISOR', 'TEACHER', 'STUDENT'];
const VALID_PERMISSIONS = [
  'CREATE_STUDENT',
  'EDIT_STUDENT',
  'CREATE_TEACHER',
  'EDIT_TEACHER',
  'MANAGE_CLASSES',
  'MANAGE_TIMETABLE',
  'MANAGE_RESOURCES',
  'MANAGE_TESTS',
  'MANAGE_ANNOUNCEMENTS',
  'VIEW_PAYMENTS',
  'MANAGE_PAYMENTS',
  'MANAGE_SETTINGS',
];
const RESOURCE_TYPES = ['Notes', 'PDF', 'Assignment', 'Worksheet', 'Question paper', 'Revision material', 'Reference material'];
const ALLOWED_FILES: Record<string, string[]> = {
  'application/pdf': ['.pdf'],
  'image/jpeg': ['.jpg', '.jpeg'],
  'image/png': ['.png'],
  'text/plain': ['.txt'],
};
const MAX_FILE_SIZE = 10 * 1024 * 1024;
const PRIVATE_UPLOAD_DIR = path.resolve(process.cwd(), 'private-uploads');

app.use(cors({ origin: config.clientUrl, credentials: true }));
app.use(express.json({ limit: '12mb', verify: (req, _res, buffer) => { (req as any).rawBody = buffer; } }));

const rateLimitMiddleware = (req: express.Request, res: express.Response, next: express.NextFunction) => {
  const ip = req.ip || 'unknown';
  const current = loginAttempts.get(ip) || { count: 0, firstAttempt: Date.now() };

  if (Date.now() - current.firstAttempt > 60 * 1000) {
    loginAttempts.set(ip, { count: 1, firstAttempt: Date.now() });
    return next();
  }

  if (current.count >= 5) {
    return res.status(429).json({ message: 'Too many login attempts. Please try again later.' });
  }

  loginAttempts.set(ip, { count: current.count + 1, firstAttempt: current.firstAttempt });
  next();
};

const getTokenFromRequest = (req: express.Request): string | null => {
  const cookieHeader = req.headers.cookie;
  if (cookieHeader) {
    const cookieItems = cookieHeader.split(';').map((entry) => entry.trim());
    const tokenCookie = cookieItems.find((entry) => entry.startsWith('mma_token='));
    if (tokenCookie) {
      return decodeURIComponent(tokenCookie.split('=')[1] || '');
    }
  }
};

```

```

    }
  }
  const authHeader = req.headers.authorization;
  if (authHeader && authHeader.startsWith('Bearer ')) return authHeader.replace('Bearer ', '');
  return null;
};

const setAuthCookie = (res: express.Response, token: string) => {
  res.setHeader('Set-Cookie', `mma_token=${encodeURIComponent(token)}; Max-Age=604800; Path=/; HttpOnly; SameSite=Lax`);
};

const authMiddleware = async (req: express.Request, res: express.Response, next: express.NextFunction) => {
  const token = getTokenFromRequest(req);
  if (!token) return res.status(401).json({ message: 'Unauthorized' });

  try {
    const payload = verifyToken(token);
    const user = await prisma.user.findUnique({ where: { id: payload.userId } });
    if (!user || !user.isActive) {
      return res.status(403).json({ message: 'Account is disabled or unavailable.' });
    }

    (req as any).user = {
      userId: user.id,
      email: user.email,
      fullName: user.fullName,
      role: user.role,
      status: user.status,
    };
    next();
  } catch (_error) {
    return res.status(401).json({ message: 'Invalid or expired session.' });
  }
};

const requireRoles = (...roles: string[]) => {
  return (req: express.Request, res: express.Response, next: express.NextFunction) => {
    const userRole = (req as any).user?.role;
    if (!roles.includes(userRole)) {
      return res.status(403).json({ message: 'Permission denied.' });
    }
    next();
  };
};

const requirePermission = (permission: string) => {
  return async (req: express.Request, res: express.Response, next: express.NextFunction) => {
    const userRole = (req as any).user?.role;
    if (userRole === 'HEAD') return next();
    if (userRole !== 'SUPERVISOR') {
      return res.status(403).json({ message: 'Permission denied.' });
    }
  };

  const supervisorPermission = await prisma.supervisorPermission.findFirst({
    where: {
      userId: (req as any).user.userId,
      permission,
      isActive: true,
    },
  });
};

```

```

    if (!supervisorPermission) {
      return res.status(403).json({ message: 'Supervisor permission missing.' });
    }

    next();
  };

  const loginSchema = z.object({
    email: z.string().email(),
    password: z.string().min(6),
  });

  const enquirySchema = z.object({
    studentName: z.string().min(2),
    parentName: z.string().min(2),
    phone: z.string().min(8),
    email: z.string().email(),
    className: z.string().min(1),
    subject: z.string().min(1),
    batch: z.string().min(1),
    message: z.string().min(10),
  });

  const studentSchema = z.object({
    email: z.string().email(),
    fullName: z.string().min(2),
    className: z.string().min(1),
    batch: z.string().min(1),
    subjects: z.string().min(1),
    guardianName: z.string().min(2),
    guardianPhone: z.string().min(8),
    password: z.string().min(8),
  });

  const teacherSchema = z.object({
    email: z.string().email(),
    fullName: z.string().min(2),
    classes: z.string().min(1),
    subjects: z.string().min(1),
    batches: z.string().min(1),
    password: z.string().min(8),
  });

  const resourceSchema = z.object({
    title: z.string().min(2).max(120),
    description: z.string().max(1000).optional().default(''),
    className: z.string().min(1).max(80),
    subject: z.string().min(1).max(80),
    topic: z.string().min(1).max(120),
    type: z.enum(['Notes', 'PDF', 'Assignment', 'Worksheet', 'Question paper', 'Revision material', 'Reference material']),
    fileName: z.string().min(1).max(180),
    mimeType: z.string().min(1),
    fileData: z.string().min(1),
  });

  const testSchema = z.object({
    title: z.string().min(2).max(120),
    className: z.string().min(1),
    subject: z.string().min(1),
    topic: z.string().min(1),
  });

```

```

    date: z.string().min(1),
    time: z.string().min(1),
    instructions: z.string().max(1000).optional().default(''),
  });

const announcementSchema = z.object({
  title: z.string().min(2).max(120),
  message: z.string().min(2).max(2000),
  audience: z.enum(['ALL', 'STUDENT', 'TEACHER', 'CLASS']),
  className: z.string().optional(),
  subject: z.string().optional(),
});

const timetableSchema = z.object({
  day: z.string().min(1),
  startTime: z.string().min(1),
  endTime: z.string().min(1),
  classLevel: z.string().min(1),
  subject: z.string().min(1),
  teacherId: z.string().optional(),
  batch: z.string().optional(),
  room: z.string().optional(),
});

const paymentOrderSchema = z.object({
  studentFeeId: z.string().min(1),
  amountPaise: z.number().int().positive(),
  idempotencyKey: z.string().min(16).max(120),
});

const paymentVerifySchema = z.object({
  paymentRecordId: z.string().min(1),
  razorpayOrderId: z.string().min(1),
  razorpayPaymentId: z.string().min(1),
  razorpaySignature: z.string().min(1),
});

const feeStructureSchema = z.object({
  name: z.string().min(2).max(120),
  description: z.string().max(1000).optional(),
  className: z.string().min(1),
  totalAmountPaise: z.number().int().positive(),
});

const studentFeeSchema = z.object({
  studentId: z.string().min(1),
  feeStructureId: z.string().min(1),
});

const splitValues = (value: string | null | undefined) => (value || '').split(',').map((item) => item.trim()).filter(Boolean);

const getTeacherAssignments = async (teacherId: string) => prisma.teacherAssignment.findMany({ where: { teacherId }, orderBy: [{ className: 'asc' }, { subject: 'asc' }] });

const getStudentEnrollments = async (studentId: string) => prisma.studentEnrollment.findMany({ where: { studentId }, orderBy: [{ className: 'asc' }, { subject: 'asc' }] });

const notifyUsers = async (userIds: string[], type: string, title: string, message: string) => {
  if (!userIds.length) return;
  await prisma.notification.createMany({ data: [...new Set(userIds)].map((userId) => ({ userId, type, title, message })) });
};

```

```

};

const matchesAssignment = (assignments: Array<{ className: string; subject: string }>, className: string, subject: string) =>
  assignments.some((assignment) => assignment.className === className && assignment.subject.toLowerCase() === subject.toLowerCase());

const razorpayRequest = async (endpoint: string, method: string, body?: unknown) => {
  if (!config.paymentKeyId || !config.paymentKeySecret) throw new Error('Payment gateway is not configured.');
```

```

  const response = await fetch('https://api.razorpay.com/v1${endpoint}', {
    method,
    headers: {
      Authorization: 'Basic ${Buffer.from(`${config.paymentKeyId}:${config.paymentKeySecret}`).toString('base64')}',
      'Content-Type': 'application/json',
    },
    ...(body ? { body: JSON.stringify(body) } : {}),
  });
  const payload = await response.json() as Record<string, unknown>;
  if (!response.ok) throw new Error(String(payload.error || 'Payment gateway request failed.));
  return payload;
};

const paymentSummary = (fee: { totalAmountPaise: number; paidAmountPaise: number }) => ({
  totalAmountPaise: fee.totalAmountPaise,
  paidAmountPaise: fee.paidAmountPaise,
  outstandingAmountPaise: Math.max(0, fee.totalAmountPaise - fee.paidAmountPaise),
  status: fee.paidAmountPaise >= fee.totalAmountPaise ? 'PAID' : fee.paidAmountPaise > 0 ? 'PARTIALLY_PAID' : 'OUTSTANDING',
});

const createReceipt = async (tx: any, paymentId: string) => {
  const payment = await tx.payment.findUnique({ where: { id: paymentId }, include: { student: { include: { user: true } }, studentFee: { include: { feeStructure: true } } } });
  if (!payment || payment.status !== 'SUCCESS') return null;
  return tx.paymentReceipt.upsert({
    where: { paymentId },
    update: {},
    create: {
      paymentId,
      receiptNumber: `MMA-${new Date().getFullYear()}-${randomUUID().slice(0, 8).toUpperCase()}`,
      studentName: payment.student.user.fullName,
      studentIdentifier: payment.student.studentId,
      className: payment.student.className,
      amountPaise: payment.amountPaise,
      currency: payment.currency,
      feeDescription: payment.studentFee.feeStructure.description || payment.studentFee.feeStructure.name,
      status: payment.status,
    },
  });
};

const settleSuccessfulPayment = async (paymentId: string, gatewayPaymentId: string, gatewaySignature?: string) =>
  > prisma.$transaction(async (tx) => {
    const payment = await tx.payment.findUnique({ where: { id: paymentId }, include: { studentFee: true } });
    if (!payment) throw new Error('Payment record not found.');
```

```

    if (payment.status === 'SUCCESS') return payment;
    const outstanding = payment.studentFee.totalAmountPaise - payment.studentFee.paidAmountPaise;
    if (payment.amountPaise > outstanding) throw new Error('Payment exceeds the outstanding balance.');
```

```

    const updatedPayment = await tx.payment.update({ where: { id: payment.id }, data: { status: 'SUCCESS', gatewayPaymentId, gatewaySignature } });
  });

```

```

    const paidAmountPaise = Math.min(payment.studentFee.totalAmountPaise, payment.studentFee.paidAmountPaise + payment.amountPaise);
    await tx.studentFee.update({ where: { id: payment.studentFeeId }, data: { paidAmountPaise, status: paidAmountPaise >= payment.studentFee.totalAmountPaise ? 'PAID' : 'PARTIALLY_PAID' } });
    await createReceipt(tx, payment.id);
    await tx.auditLog.create({ data: { userId: payment.createdById, userRole: 'STUDENT', action: 'PAYMENT_VERIFIED', targetType: 'Payment', targetId: payment.id, details: 'Verified gateway payment ${gatewayPaymentId}' } });
    return updatedPayment;
  });

app.get('/api/health', async (_req, res) => {
  try {
    await prisma.$queryRaw`SELECT 1`;
    res.json({ ok: true, message: 'MMA API is running', timestamp: new Date().toISOString() });
  } catch (_error) {
    res.status(500).json({ message: 'Database connection failed' });
  }
});

app.get('/api/academy', async (_req, res) => {
  const settings = await prisma.academySettings.findFirst();
  res.json({ data: settings || { siteName: 'Manish Mishra Academy', tagline: 'Learn Better. Think Smarter. Achieve More.' } });
});

app.get('/api/courses', async (_req, res) => {
  const courses = await prisma.course.findMany({ orderBy: { createdAt: 'asc' } });
  res.json({ data: courses });
});

app.get('/api/classes', async (_req, res) => {
  const classes = await prisma.course.findMany({ orderBy: { name: 'asc' } });
  res.json({ data: classes });
});

app.get('/api/timetable', async (_req, res) => {
  const timetable = await prisma.timetable.findMany({ orderBy: { day: 'asc' } });
  res.json({ data: timetable });
});

app.get('/api/facilities', async (_req, res) => {
  const facilities = await prisma.facility.findMany({ orderBy: { createdAt: 'asc' } });
  res.json({ data: facilities });
});

app.get('/api/testimonials', async (_req, res) => {
  const testimonials = await prisma.testimonial.findMany({ where: { isActive: true } });
  res.json({ data: testimonials });
});

app.post('/api/enquiries', async (req, res) => {
  const result = enquirySchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ message: 'Please fill in all required fields correctly.', errors: result.error.flatten().fieldErrors });
  }

  try {
    const enquiry = await prisma.enquiry.create({ data: result.data });
    res.status(201).json({ message: 'Thank you for contacting Manish Mishra Academy. Our team will get back to you soon.', data: enquiry });
  } catch (_error) {

```

```

    res.status(500).json({ message: 'Something went wrong. Please try again.' });
  }
});

app.post('/api/auth/login', rateLimitMiddleware, async (req, res) => {
  const result = loginSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ message: 'Please provide a valid email and password.' });
  }

  const user = await prisma.user.findUnique({ where: { email: result.data.email } });
  if (!user || !user.isActive) {
    return res.status(401).json({ message: 'Invalid email or password.' });
  }

  const passwordMatch = await bcrypt.compare(result.data.password, user.password);
  if (!passwordMatch) {
    return res.status(401).json({ message: 'Invalid email or password.' });
  }

  if (!VALID_ROLES.includes(user.role)) {
    return res.status(403).json({ message: 'Account role is not recognized.' });
  }

  loginAttempts.delete(req.ip || 'unknown');
  const token = signToken({ userId: user.id, email: user.email, role: user.role });
  setAuthCookie(res, token);

  const redirectMap = {
    HEAD: '/admin/dashboard',
    SUPERVISOR: '/admin/dashboard',
    TEACHER: '/teacher/dashboard',
    STUDENT: '/student/dashboard',
  } as const;

  res.json({
    user: { id: user.id, email: user.email, fullName: user.fullName, role: user.role, status: user.status },
    redirectTo: redirectMap[user.role as keyof typeof redirectMap] || '/',
  });
});

app.post('/api/auth/logout', (req, res) => {
  res.setHeader('Set-Cookie', 'mma_token=; Max-Age=0; Path=/; HttpOnly; SameSite=Lax');
  res.json({ message: 'Logged out successfully.' });
});

app.get('/api/auth/me', authMiddleware, async (req, res) => {
  const user = await prisma.user.findUnique({ where: { id: (req as any).user.userId } });
  if (!user) {
    return res.status(404).json({ message: 'User not found.' });
  }

  res.json({ user: { id: user.id, email: user.email, fullName: user.fullName, role: user.role, status: user.status } });
});

app.get('/api/admin/dashboard', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (_req, res) => {
  const [students, teachers, enquiries, courses, resources] = await Promise.all([
    prisma.student.count(),
    prisma.teacher.count(),
    prisma.enquiry.count(),
    prisma.course.count(),
  ]);
});

```

```

    prisma.resource.count(),
  ]);

  res.json({ data: { students, teachers, enquiries, courses, resources } });
});

app.get('/api/admin/students', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (req, res) => {
  if ((req as any).user.role === 'SUPERVISOR') {
    const hasPermission = await prisma.supervisorPermission.findFirst({
      where: { userId: (req as any).user.userId, permission: 'EDIT_STUDENT', isActive: true },
    });
    if (!hasPermission) {
      return res.status(403).json({ message: 'Supervisor does not have student management permission.' });
    }
  }

  const students = await prisma.student.findMany({
    include: { user: true },
    orderBy: { createdAt: 'desc' },
  });

  res.json({ data: students });
});

app.post('/api/admin/students', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('CREATE_STUDENT'), async (req, res) => {
  const result = studentSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ message: 'Invalid student details.', errors: result.error.flatten().fieldErrors });
  }

  const userExists = await prisma.user.findUnique({ where: { email: result.data.email } });
  if (userExists) {
    return res.status(409).json({ message: 'A user with this email already exists.' });
  }

  const passwordHash = await bcrypt.hash(result.data.password, 10);
  const user = await prisma.user.create({
    data: {
      email: result.data.email,
      password: passwordHash,
      fullName: result.data.fullName,
      role: 'STUDENT',
      status: 'ACTIVE',
      isActive: true,
    },
  });

  const student = await prisma.student.create({
    data: {
      userId: user.id,
      className: result.data.className,
      batch: result.data.batch,
      subjects: result.data.subjects,
      guardianName: result.data.guardianName,
      guardianPhone: result.data.guardianPhone,
      accountStatus: 'ACTIVE',
    },
  });

  await prisma.auditLog.create({

```

```

    data: {
      userId: (req as any).user.userId,
      userRole: (req as any).user.role,
      action: 'CREATE_STUDENT',
      targetType: 'Student',
      targetId: student.id,
      details: 'Created student ${result.data.fullName}',
    },
  });

  res.status(201).json({ message: 'Student created successfully.', data: student });
});

app.get('/api/admin/teachers', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (req, res) => {
  if ((req as any).user.role === 'SUPERVISOR') {
    const hasPermission = await prisma.supervisorPermission.findFirst({
      where: { userId: (req as any).user.userId, permission: 'CREATE_TEACHER', isActive: true },
    });
    if (!hasPermission) {
      return res.status(403).json({ message: 'Supervisor does not have teacher management permission.' });
    }
  }

  const teachers = await prisma.teacher.findMany({
    include: { user: true },
    orderBy: { createdAt: 'desc' },
  });

  res.json({ data: teachers });
});

app.post('/api/admin/teachers', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('CREATE_TEACHER'), async (req, res) => {
  const result = teacherSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({ message: 'Invalid teacher details.', errors: result.error.flatten().fieldErrors });
  }

  const userExists = await prisma.user.findUnique({ where: { email: result.data.email } });
  if (userExists) {
    return res.status(409).json({ message: 'A user with this email already exists.' });
  }

  const passwordHash = await bcrypt.hash(result.data.password, 10);
  const user = await prisma.user.create({
    data: {
      email: result.data.email,
      password: passwordHash,
      fullName: result.data.fullName,
      role: 'TEACHER',
      status: 'ACTIVE',
      isActive: true,
    },
  });

  const teacher = await prisma.teacher.create({
    data: {
      userId: user.id,
      classes: result.data.classes,
      subjects: result.data.subjects,
      batches: result.data.batches,
    },
  });

```

```

        accountStatus: 'ACTIVE',
      },
    });

    await prisma.auditLog.create({
      data: {
        userId: (req as any).user.userId,
        userRole: (req as any).user.role,
        action: 'CREATE_TEACHER',
        targetType: 'Teacher',
        targetId: teacher.id,
        details: 'Created teacher ${result.data.fullName}',
      },
    });

    res.status(201).json({ message: 'Teacher created successfully.', data: teacher });
  });

  app.get('/api/teacher/dashboard', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
    const user = await prisma.user.findUnique({ where: { id: (req as any).user.userId }, include: { teacher: true } });
    const assignments = user?.teacher ? await getTeacherAssignments(user.teacher.id) : [];
    const timetable = user?.teacher ? await prisma.timetable.findMany({ where: { teacherId: user.teacher.id }, take: 5, orderBy: { day: 'asc' } }) : [];
    const resources = user?.teacher ? await prisma.resource.findMany({ where: { teacherId: user.teacher.id }, take: 3, orderBy: { createdAt: 'desc' } }) : [];

    res.json({ data: { user, assignments, timetable, resources } });
  });

  app.get('/api/student/dashboard', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
    const user = await prisma.user.findUnique({ where: { id: (req as any).user.userId }, include: { student: true } });
    const enrollments = user?.student ? await getStudentEnrollments(user.student.id) : [];
    const classNames = [...new Set(enrollments.map((item) => item.className))];
    const subjects = [...new Set(enrollments.map((item) => item.subject))];
    const feeStatus = await prisma.feeRecord.findFirst({ where: { studentName: user?.fullName || '' } });
    const announcements = await prisma.announcement.findMany({ where: { OR: [{ audience: 'ALL' }, { audience: 'STUDENT' }, ...classNames.map((className) => ({ audience: 'CLASS', className })) ] }, take: 3, orderBy: { createdAt: 'desc' } });
    const tests = await prisma.testRecord.findMany({ where: { OR: classNames.flatMap((className) => subjects.map((subject) => ({ className, subject }))) }, take: 3, orderBy: { createdAt: 'desc' } });
    const notifications = user ? await prisma.notification.findMany({ where: { userId: user.id }, take: 5, orderBy: { createdAt: 'desc' } }) : [];
    const teachers = user?.student ? await prisma.teacherAssignment.findMany({ where: { className: { in: classNames }, subject: { in: subjects }, include: { teacher: { include: { user: { select: { fullName: true } } } } }, distinct: ['teacherId'] }) : [];

    res.json({ data: { user, enrollments, assignedTeachers: teachers, feeStatus, announcements, tests, notifications } });
  });

  app.get('/api/student/fees', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
    const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
    if (!student) return res.status(404).json({ message: 'Student profile not found.' });
    const fees = await prisma.studentFee.findMany({ where: { studentId: student.id }, include: { feeStructure: true }, payments: { include: { receipt: true }, orderBy: { createdAt: 'desc' } } }, orderBy: { createdAt: 'desc' });
    ;
    res.json({ data: fees.map((fee) => ({ ...fee, summary: paymentSummary(fee), payments: fee.payments.map((payment) => ({ ...payment, receipt: payment.receipt ? { id: payment.receipt.id, receiptNumber: payment.receipt.receiptNumber, issuedAt: payment.receipt.issuedAt } : null }))) }));
  });

```

```

app.post('/api/student/payments/order', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const result = paymentOrderSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid payment amount.' });
  const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
  if (!student) return res.status(404).json({ message: 'Student profile not found.' });
  const fee = await prisma.studentFee.findFirst({ where: { id: result.data.studentFeeId, studentId: student.id }
, include: { feeStructure: true } });
  if (!fee) return res.status(404).json({ message: 'Fee record not found.' });
  const outstanding = Math.max(0, fee.totalAmountPaise - fee.paidAmountPaise);
  if (!outstanding || result.data.amountPaise > outstanding) return res.status(400).json({ message: 'Payment amo
unt exceeds the outstanding balance.' });

  const existingPayment = await prisma.payment.findUnique({ where: { idempotencyKey: result.data.idempotencyKey
} });
  if (existingPayment) {
    if (existingPayment.studentId !== student.id || existingPayment.studentFeeId !== fee.id || existingPayment.a
mountPaise !== result.data.amountPaise) return res.status(409).json({ message: 'Idempotency key is already assoc
iated with another payment attempt.' });
    return res.status(200).json({ data: { paymentRecordId: existingPayment.id, orderId: existingPayment.gatewayO
rderId, amountPaise: existingPayment.amountPaise, currency: existingPayment.currency, keyId: config.paymentKeyId
|| null, sandbox: config.paymentSandbox } });
  }
  const idempotencyKey = result.data.idempotencyKey;
  const gatewayOrder = config.paymentKeyId && config.paymentKeySecret ? await razorpayRequest('/orders', 'POST',
{ amount: result.data.amountPaise, currency: 'INR', receipt: idempotencyKey, notes: { studentFeeId: fee.id, san
dbox: config.paymentSandbox } }) : { id: 'sandbox_order_${randomUUID()}' };
  const payment = await prisma.payment.create({ data: { studentId: student.id, studentFeeId: fee.id, amountPaise
: result.data.amountPaise, feeType: fee.feeStructure.name, gatewayOrderId: String(gatewayOrder.id), idempotencyK
ey, createdById: (req as any).user.userId, isSandbox: config.paymentSandbox, metadataJson: JSON.stringify({ gate
wayOrder }) } });
  res.status(201).json({ data: { paymentRecordId: payment.id, orderId: payment.gatewayOrderId, amountPaise: paym
ent.amountPaise, currency: payment.currency, keyId: config.paymentKeyId || null, sandbox: config.paymentSandbox
} });
});

app.post('/api/student/payments/verify', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const result = paymentVerifySchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid payment verification details.' });
  const payment = await prisma.payment.findFirst({ where: { id: result.data.paymentRecordId, student: { userId:
(req as any).user.userId }, gatewayOrderId: result.data.razorpayOrderId } });
  if (!payment) return res.status(404).json({ message: 'Payment attempt not found.' });
  const expectedSignature = createHmac('sha256', config.paymentKeySecret || 'development-payment-secret').update
(`${result.data.razorpayOrderId}|${result.data.razorpayPaymentId}`).digest('hex');
  if (expectedSignature !== result.data.razorpaySignature) {
    await prisma.payment.update({ where: { id: payment.id }, data: { status: 'FAILED', failureReason: 'Invalid g
ateway signature' } });
    return res.status(400).json({ message: 'Payment could not be verified.' });
  }
  const settled = await settleSuccessfulPayment(payment.id, result.data.razorpayPaymentId, result.data.razorpayS
ignature);
  res.json({ message: 'Payment verified successfully.', data: settled });
});

app.post('/api/payments/webhook', async (req, res) => {
  const signature = req.headers['x-razorpay-signature'];
  const rawBody = (req as any).rawBody as Buffer | undefined;
  if (!config.paymentWebhookSecret || !signature || !rawBody) return res.status(400).json({ message: 'Invalid we
bhook request.' });
  const expectedSignature = createHmac('sha256', config.paymentWebhookSecret).update(rawBody).digest('hex');
  if (expectedSignature !== signature) return res.status(400).json({ message: 'Invalid webhook signature.' });
  const event = req.body as { event?: string; payload?: { payment?: { entity?: { id?: string; order_id?: string;

```

```

error_description?: string } } } };
const entity = event.payload?.payment?.entity;
if (!entity?.order_id) return res.json({ received: true });
const payment = await prisma.payment.findUnique({ where: { gatewayOrderId: entity.order_id } });
if (!payment) return res.json({ received: true });
if (event.event === 'payment.captured' && entity.id) await settleSuccessfulPayment(payment.id, entity.id);
if (event.event === 'payment.failed' && payment.status !== 'SUCCESS') await prisma.payment.update({ where: { id: payment.id }, data: { status: 'FAILED', failureReason: entity.error_description || 'Gateway payment failed.' } });
res.json({ received: true });
});

app.get('/api/student/receipts/:id', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const receipt = await prisma.paymentReceipt.findFirst({ where: { id: String(req.params.id), payment: { student: { userId: (req as any).user.userId } } }, include: { payment: { include: { studentFee: { include: { feeStructure: true } } } } } });
  if (!receipt) return res.status(404).json({ message: 'Receipt not found.' });
  res.json({ data: receipt });
});

app.get('/api/student/receipts/:id/download', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const receipt = await prisma.paymentReceipt.findFirst({ where: { id: String(req.params.id), payment: { student: { userId: (req as any).user.userId } } }, include: { payment: { include: { studentFee: { include: { feeStructure: true } } } } } });
  if (!receipt) return res.status(404).json({ message: 'Receipt not found.' });
  const escapeHtml = (value: string) => value.replace(/<"/g, (character) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '": '&quot;', '": '&#39;' }[character] || character));
  const html = `<!doctype html><html><head><meta charset="utf-8"><title>MMA Receipt ${escapeHtml(receipt.receiptNumber)}</title><style>body{font-family:Arial,sans-serif;max-width:720px;margin:40px auto;color:#172033}h1{color:#0369a1}.row{display:flex;justify-content:space-between;border-bottom:1px solid #ddd;padding:12px 0}.muted{color:#64748b}</style></head><body><h1>MANISH MISHRA ACADEMY (MMA)</h1><p class="muted">Payment receipt</p><div class="row"><strong>Student</strong><span>${escapeHtml(receipt.studentName)}</span></div><div class="row"><strong>Student ID</strong><span>${escapeHtml(receipt.studentIdentifier)}</span></div><div class="row"><strong>Class</strong><span>${escapeHtml(receipt.className || 'Not specified')}</span></div><div class="row"><strong>Receipt</strong><span>${escapeHtml(receipt.receiptNumber)}</span></div><div class="row"><strong>Payment status</strong><span>${escapeHtml(receipt.status)}</span></div><div class="row"><strong>Amount</strong><span>${escapeHtml(receipt.currency)} ${((receipt.amountPaise / 100).toFixed(2))}</span></div><div class="row"><strong>Fee description</strong><span>${escapeHtml(receipt.feeDescription)}</span></div><div class="row"><strong>Issued</strong><span>${escapeHtml(receipt.issuedAt.toISOString())}</span></div></body></html>`;
  res.setHeader('Content-Type', 'text/html; charset=utf-8');
  res.setHeader('Content-Disposition', 'attachment; filename="${receipt.receiptNumber}.html"');
  res.send(html);
});

app.get('/api/admin/fees', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (req, res) => {
  const permission = (req as any).user.role === 'HEAD' ? true : await prisma.supervisorPermission.findFirst({ where: { userId: (req as any).user.userId, permission: 'VIEW_PAYMENTS', isActive: true } });
  if (!permission) return res.status(403).json({ message: 'Supervisor payment permission missing.' });
  const fees = await prisma.studentFee.findMany({ include: { student: { include: { user: true } }, feeStructure: true }, orderBy: { updatedAt: 'desc' } });
  res.json({ data: fees.map((fee) => ({ ...fee, summary: paymentSummary(fee) })) });
});

app.get('/api/admin/fee-structures', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('MANAGE_PAYMENTS'), async (_req, res) => {
  res.json({ data: await prisma.feeStructure.findMany({ where: { isActive: true }, orderBy: { createdAt: 'desc' } }) });
});

app.post('/api/admin/fee-structures', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('MANAGE_PAYMENTS'), async (req, res) => {
  const result = feeStructureSchema.safeParse(req.body);

```

```

    if (!result.success) return res.status(400).json({ message: 'Invalid fee structure.' });
    const structure = await prisma.feeStructure.create({ data: { ...result.data, createdById: (req as any).user.userId } });
    res.status(201).json({ data: structure });
  });

app.post('/api/admin/student-fees', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('MANAGE_PAYMENTS'), async (req, res) => {
  const result = studentFeeSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid student fee assignment.' });
  const [student, structure] = await Promise.all([prisma.student.findUnique({ where: { id: result.data.studentId } }), prisma.feeStructure.findUnique({ where: { id: result.data.feeStructureId } })]);
  if (!student || !structure) return res.status(404).json({ message: 'Student or fee structure not found.' });
  const fee = await prisma.studentFee.upsert({ where: { studentId_feeStructureId: result.data }, update: { totalAmountPaise: structure.totalAmountPaise }, create: { studentId: student.id, feeStructureId: structure.id, totalAmountPaise: structure.totalAmountPaise } });
  res.status(201).json({ data: fee });
});

app.get('/api/admin/payments', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (req, res) => {
  const permission = (req as any).user.role === 'HEAD' ? true : await prisma.supervisorPermission.findFirst({ where: { userId: (req as any).user.userId, permission: 'VIEW_PAYMENTS', isActive: true } });
  if (!permission) return res.status(403).json({ message: 'Supervisor payment permission missing.' });
  const search = String(req.query.search || '').trim();
  const className = String(req.query.className || '').trim();
  const from = String(req.query.from || '').trim();
  const to = String(req.query.to || '').trim();
  const payments = await prisma.payment.findMany({ where: { AND: [ ...(req.query.status ? [{ status: String(req.query.status) } ] : []), ...(req.query.paymentId ? [{ gatewayPaymentId: String(req.query.paymentId) } ] : []), ...(search ? [{ student: { OR: [{ studentId: { contains: search } }, { user: { fullName: { contains: search } } } ] } ] : []), ...(className ? [{ student: { className } } ] : []), ...(from || to ? [{ createdAt: { ...(from ? { gt: new Date(from) } : {}), ...(to ? { lte: new Date(`${to}T23:59:59.999Z`) } : {} ) } ] : [] ) } ], include: { student: { include: { user: true } }, receipt: true }, orderBy: { createdAt: 'desc' } });
  const totals = payments.reduce((result, payment) => { result[payment.status] = (result[payment.status] || 0) + payment.amountPaise; return result; }, {} as Record<string, number>);
  res.json({ data: payments, totals });
});

app.get('/api/teacher/assignments', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  if (!teacher) return res.status(404).json({ message: 'Teacher profile not found.' });
  res.json({ data: await getTeacherAssignments(teacher.id) });
});

app.get('/api/student/resources', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
  if (!student) return res.status(404).json({ message: 'Student profile not found.' });
  const enrollments = await getStudentEnrollments(student.id);
  const search = String(req.query.search || '').trim();
  const className = String(req.query.className || '').trim();
  const subject = String(req.query.subject || '').trim();
  const type = String(req.query.type || '').trim();
  const teacherId = String(req.query.teacherId || '').trim();
  if (!enrollments.length) return res.json({ data: [], filters: { classes: [], subjects: [], types: RESOURCE_TYPES } });
  const resources = await prisma.resource.findMany({
    where: {
      AND: [
        { OR: enrollments.map((enrollment) => ({ className: enrollment.className, subject: enrollment.subject })) },
        ...(className ? [{ className } ] : []),
        ...(subject ? [{ subject } ] : []),
      ],
    },
  });

```

```

        ...(type ? [{ type }] : []),
        ...(teacherId ? [{ teacherId }] : []),
        ...(search ? [{ OR: [{ title: { contains: search } }, { description: { contains: search } }, { topic: {
contains: search } } ] ] : []),
    ],
  },
  include: { teacher: { include: { user: { select: { fullName: true } } } } },
  orderBy: { createdAt: String(req.query.order || '') === 'oldest' ? 'asc' : 'desc' },
});
res.json({ data: resources, filters: { classes: [...new Set(enrollments.map((item) => item.className))], subjects: [...new Set(enrollments.map((item) => item.subject))], types: RESOURCE_TYPES } });
});

app.get('/api/student/teachers', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
  const enrollments = student ? await getStudentEnrollments(student.id) : [];
  const teachers = await prisma.teacherAssignment.findMany({ where: { OR: enrollments.map((item) => ({ className: item.className, subject: item.subject })), include: { teacher: { include: { user: { select: { fullName: true, email: true } } } } } });
  res.json({ data: teachers });
});

app.get('/api/teacher/resources', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  if (!teacher) return res.status(404).json({ message: 'Teacher profile not found.' });
  const search = String(req.query.search || '').trim();
  const resources = await prisma.resource.findMany({
    where: { teacherId: teacher.id, ...(search ? { OR: [{ title: { contains: search } }, { topic: { contains: search } } ] : {} ) },
    include: { teacher: { include: { user: { select: { fullName: true } } } } },
    orderBy: { createdAt: 'desc' },
  });
  res.json({ data: resources, assignments: await getTeacherAssignments(teacher.id), types: RESOURCE_TYPES });
});

app.post('/api/teacher/resources', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const result = resourceSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid resource details.', errors: result.error.flatten().fieldErrors });

  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  if (!teacher) return res.status(404).json({ message: 'Teacher profile not found.' });
  const assignments = await getTeacherAssignments(teacher.id);
  if (!matchesAssignment(assignments, result.data.className, result.data.subject)) {
    return res.status(403).json({ message: 'You may only publish resources to your assigned class and subject.' });
  }

  const extension = path.extname(result.data.fileName).toLowerCase();
  const allowedExtensions = ALLOWED_FILES[result.data.mimeType];
  if (!allowedExtensions || !allowedExtensions.includes(extension)) return res.status(400).json({ message: 'Unsupported or unsafe file type.' });
  let fileBuffer: Buffer;
  try { fileBuffer = Buffer.from(result.data.fileData.replace(/^data:[^;]+;base64/, ''), 'base64'); } catch (_error) { return res.status(400).json({ message: 'Invalid file data.' }); }
  if (!fileBuffer.length || fileBuffer.length > MAX_FILE_SIZE) return res.status(400).json({ message: 'File must be between 1 byte and 10 MB.' });

  await mkdir(PRIVATE_UPLOAD_DIR, { recursive: true });
  const storedFileName = `${randomUUID()}${extension}`;
  await writeFile(path.join(PRIVATE_UPLOAD_DIR, storedFileName), fileBuffer, { flag: 'wx' });
  const resource = await prisma.resource.create({ data: { title: result.data.title, description: result.data.description,

```

```

cription, className: result.data.className, subject: result.data.subject, topic: result.data.topic, type: result
.data.type, fileName: result.data.fileName, storedFileName, mimeType: result.data.mimeType, fileSize: fileBuffer
.length, teacherId: teacher.id, createdById: (req as any).user.userId } }));

    const students = await prisma.student.findMany({ where: { enrollments: { some: { className: result.data.classN
ame, subject: result.data.subject } } }, select: { userId: true } });
    await notifyUsers(students.map((student) => student.userId), 'NEW_RESOURCE', 'New study resource', `${result.d
ata.title} is available for ${result.data.className} ${result.data.subject}.');
    res.status(201).json({ message: 'Resource published successfully.', data: resource });
  });

app.get('/api/resources/:id/download', authMiddleware, async (req, res) => {
  const resource = await prisma.resource.findUnique({ where: { id: String(req.params.id) }, include: { teacher:
true } });
  if (!resource) return res.status(404).json({ message: 'Resource not found.' });
  const user = (req as any).user;
  let allowed = user.role === 'HEAD' || user.role === 'SUPERVISOR' || (user.role === 'TEACHER' && resource.teach
er.userId === user.userId);
  if (user.role === 'STUDENT') {
    const student = await prisma.student.findUnique({ where: { userId: user.userId } });
    const enrollment = student && await prisma.studentEnrollment.findFirst({ where: { studentId: student.id, cla
ssName: resource.className, subject: resource.subject } });
    allowed = Boolean(enrollment);
  }
  if (!allowed) return res.status(403).json({ message: 'You are not authorized to download this resource.' });
  res.download(path.join(PRIVATE_UPLOAD_DIR, resource.storedFileName), resource.fileName);
});

app.get('/api/teacher/timetable', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  res.json({ data: teacher ? await prisma.timetable.findMany({ where: { teacherId: teacher.id }, orderBy: { day:
'asc' } }) : [] });
});

app.get('/api/student/timetable', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
  const enrollments = student ? await getStudentEnrollments(student.id) : [];
  if (!enrollments.length) return res.json({ data: [] });
  res.json({ data: await prisma.timetable.findMany({ where: { OR: enrollments.map((item) => ({ classLevel: item.
className, subject: item.subject })) }, orderBy: { day: 'asc' } }) });
});

app.get('/api/student/notifications', authMiddleware, requireRoles('STUDENT', 'TEACHER'), async (req, res) => {
  res.json({ data: await prisma.notification.findMany({ where: { userId: (req as any).user.userId }, orderBy: {
createdAt: 'desc' }, take: 50 } ) });
});

app.post('/api/admin/announcements', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (req, res) => {
  const result = announcementSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid announcement details.' });
  const announcement = await prisma.announcement.create({ data: { ...result.data, createdById: (req as any).user
.userId } });
  const users = await prisma.user.findMany({ where: result.data.audience === 'TEACHER' ? { role: 'TEACHER' } : {
role: 'STUDENT' }, select: { id: true } });
  await notifyUsers(users.map((user) => user.id), 'ANNOUNCEMENT', result.data.title, result.data.message);
  res.status(201).json({ data: announcement });
});

app.post('/api/teacher/announcements', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const result = announcementSchema.safeParse({ ...req.body, audience: 'CLASS' });
  if (!result.success || !result.data.className || !result.data.subject) return res.status(400).json({ message:
'Class and subject are required for teacher announcements.' });

```

```

    const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
    if (!teacher || !matchesAssignment(await getTeacherAssignments(teacher.id), result.data.className, result.data.subject)) return res.status(403).json({ message: 'You may only announce to your assigned class and subject.' });
    ;
    const announcement = await prisma.announcement.create({ data: { ...result.data, createdById: (req as any).user.userId } });
    const students = await prisma.student.findMany({ where: { enrollments: { some: { className: result.data.className, subject: result.data.subject } } }, select: { userId: true } });
    await notifyUsers(students.map((student) => student.userId), 'ANNOUNCEMENT', result.data.title, result.data.message);
    res.status(201).json({ data: announcement });
  });

app.post('/api/admin/timetable', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), requirePermission('MANAGE_TIMETABLE'), async (req, res) => {
  const result = timetableSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid timetable details.' });
  const timetable = await prisma.timetable.create({ data: result.data });
  const students = await prisma.student.findMany({ where: { enrollments: { some: { className: result.data.className, subject: result.data.subject } } }, select: { userId: true } });
  await notifyUsers(students.map((student) => student.userId), 'TIMETABLE_CHANGE', 'Timetable updated', `${result.data.className} ${result.data.subject} has a timetable update.`);
  res.status(201).json({ data: timetable });
});

app.get('/api/admin/tests', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (_req, res) => {
  res.json({ data: await prisma.testRecord.findMany({ orderBy: { date: 'asc' } }) });
});

app.get('/api/teacher/tests', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  res.json({ data: teacher ? await prisma.testRecord.findMany({ where: { teacherId: teacher.id }, orderBy: { date: 'asc' } }) : [] });
});

app.post('/api/teacher/tests', authMiddleware, requireRoles('TEACHER'), async (req, res) => {
  const result = testSchema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ message: 'Invalid test details.' });
  const teacher = await prisma.teacher.findUnique({ where: { userId: (req as any).user.userId } });
  if (!teacher || !matchesAssignment(await getTeacherAssignments(teacher.id), result.data.className, result.data.subject)) return res.status(403).json({ message: 'You may only create tests for assigned classes and subjects.' });
  const test = await prisma.testRecord.create({ data: { ...result.data, teacherId: teacher.id, createdById: (req as any).user.userId } });
  const students = await prisma.student.findMany({ where: { enrollments: { some: { className: result.data.className, subject: result.data.subject } } }, select: { userId: true } });
  await notifyUsers(students.map((student) => student.userId), 'TEST_ANNOUNCEMENT', 'New test announced', `${result.data.title} is scheduled for ${result.data.date}.`);
  res.status(201).json({ data: test });
});

app.get('/api/student/tests', authMiddleware, requireRoles('STUDENT'), async (req, res) => {
  const student = await prisma.student.findUnique({ where: { userId: (req as any).user.userId } });
  const enrollments = student ? await getStudentEnrollments(student.id) : [];
  if (!enrollments.length) return res.json({ data: [] });
  res.json({ data: await prisma.testRecord.findMany({ where: { OR: enrollments.map((item) => ({ className: item.className, subject: item.subject })) }, orderBy: { date: 'asc' } }) });
});

app.get('/api/admin/audit-log', authMiddleware, requireRoles('HEAD', 'SUPERVISOR'), async (_req, res) => {
  const logs = await prisma.auditLog.findMany({ orderBy: { createdAt: 'desc' }, take: 20 });
  res.json({ data: logs });
});

```

```

});

app.get('/api/admin/permissions', authMiddleware, requireRoles('HEAD'), async (_req, res) => {
  res.json({ data: VALID_PERMISSIONS });
});

app.post('/api/admin/permissions/:userId', authMiddleware, requireRoles('HEAD'), async (req, res) => {
  const { permission, isActive } = req.body as { permission?: string; isActive?: boolean };
  if (!permission || !VALID_PERMISSIONS.includes(permission)) {
    return res.status(400).json({ message: 'Invalid permission.' });
  }

  const targetUser = await prisma.user.findUnique({ where: { id: String(req.params.userId) } });
  if (!targetUser || targetUser.role !== 'SUPERVISOR') {
    return res.status(400).json({ message: 'Permission can only be managed for supervisor accounts.' });
  }

  const existingPermission = await prisma.supervisorPermission.findFirst({
    where: { userId: targetUser.id, permission },
  });

  if (existingPermission) {
    const updatedPermission = await prisma.supervisorPermission.update({
      where: { id: existingPermission.id },
      data: { isActive: Boolean(isActive), grantedBy: (req as any).user.userId },
    });
    return res.json({ message: 'Supervisor permission updated.', data: updatedPermission });
  }

  const createdPermission = await prisma.supervisorPermission.create({
    data: {
      userId: targetUser.id,
      permission,
      grantedBy: (req as any).user.userId,
      isActive: Boolean(isActive),
    },
  });

  await prisma.auditLog.create({
    data: {
      userId: (req as any).user.userId,
      userRole: (req as any).user.role,
      action: 'PERMISSION_CHANGE',
      targetType: 'SupervisorPermission',
      targetId: createdPermission.id,
      details: 'Updated ${permission} for ${targetUser.fullName}',
    },
  });

  res.status(201).json({ message: 'Supervisor permission created.', data: createdPermission });
});

app.use((_req, res) => {
  res.status(404).json({ message: 'Route not found' });
});

app.listen(config.port, () => {
  console.log('MMA server running on http://localhost:${config.port}');
});

===== server/prisma/schema.prisma =====
generator client {

```

```

    provider = "prisma-client-js"
  }

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  password    String
  fullName    String
  role        String      @default("HEAD")
  status      String      @default("ACTIVE")
  isActive    Boolean      @default(true)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  student     Student?
  teacher     Teacher?
  auditLogs   AuditLog[]
  supervisorPermissions SupervisorPermission[]
  createdBy   String?
  createdUsers User[]       @relation("CreatedByUser")
  createdByUser User?       @relation("CreatedByUser", fields: [createdBy], references: [id])
  createdResources Resource[] @relation("ResourceCreator")
  createdAnnouncements Announcement[] @relation("AnnouncementCreator")
  createdTests TestRecord[]   @relation("TestCreator")
  notifications Notification[]
  createdFeeStructures FeeStructure[] @relation("FeeStructureCreator")
  payments    Payment[]
}

model Student {
  id          String      @id @default(cuid())
  userId      String      @unique
  user        User        @relation(fields: [userId], references: [id])
  studentId   String      @unique @default(cuid())
  className   String?
  batch       String?
  subjects    String?
  guardianName String?
  guardianPhone String?
  accountStatus String    @default("ACTIVE")
  admissionDate DateTime @default(now())
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  enrollments StudentEnrollment[]
  fees        StudentFee[]
  payments    Payment[]
}

model Teacher {
  id          String      @id @default(cuid())
  userId      String      @unique
  user        User        @relation(fields: [userId], references: [id])
  teacherId   String      @unique @default(cuid())
  classes     String?
  subjects    String?
  batches     String?
  accountStatus String    @default("ACTIVE")
  createdAt   DateTime @default(now())
}

```

```

    updatedAt      DateTime @updatedAt
    assignments     TeacherAssignment[]
    resources       Resource[]
    timetable       Timetable[]
    tests           TestRecord[]
}

model SupervisorPermission {
  id          String  @id @default(cuid())
  userId      String
  permission  String
  grantedBy   String?
  isActive    Boolean @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  user        User    @relation(fields: [userId], references: [id])

  @@unique([userId, permission])
}

model AuditLog {
  id          String  @id @default(cuid())
  userId      String?
  userRole    String?
  action      String
  targetType  String?
  targetId    String?
  details     String?
  createdAt   DateTime @default(now())
  user        User?   @relation(fields: [userId], references: [id])
}

model Resource {
  id          String  @id @default(cuid())
  title       String
  type        String
  description  String?
  className   String
  subject     String
  topic       String
  fileName    String
  storedFileName String @unique
  mimeType    String
  fileSize    Int
  teacherId   String
  createdById String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  teacher     Teacher @relation(fields: [teacherId], references: [id])
  createdBy   User    @relation("ResourceCreator", fields: [createdById], references: [id])

  @@index([className, subject])
  @@index([teacherId])
}

model Announcement {
  id          String  @id @default(cuid())
  title       String
  message     String
  audience    String  @default("ALL")
  className   String?
  subject     String?

```

```

    createdById String?
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
    createdBy   User?     @relation("AnnouncementCreator", fields: [createdById], references: [id])
}

model FeeRecord {
  id          String   @id @default(cuid())
  studentId   String
  studentName String
  amount      String
  dueDate     String?
  status      String   @default("PENDING")
  paymentStatus String @default("NOT_PAID")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model FeeStructure {
  id          String   @id @default(cuid())
  name        String
  description  String?
  className   String
  totalAmountPaise Int
  isActive    Boolean @default(true)
  createdById String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  createdBy   User     @relation("FeeStructureCreator", fields: [createdById], references: [id])
  studentFees StudentFee[]

  @@index([className, isActive])
}

model StudentFee {
  id          String   @id @default(cuid())
  studentId   String
  feeStructureId String
  totalAmountPaise Int
  paidAmountPaise Int @default(0)
  currency    String   @default("INR")
  status      String   @default("OUTSTANDING")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  student     Student  @relation(fields: [studentId], references: [id])
  feeStructure FeeStructure @relation(fields: [feeStructureId], references: [id])
  payments    Payment[]

  @@unique([studentId, feeStructureId])
  @@index([studentId, status])
}

model Payment {
  id          String   @id @default(cuid())
  studentId   String
  studentFeeId String
  amountPaise Int
  currency    String   @default("INR")
  status      String   @default("CREATED")
  feeType     String
  gateway     String   @default("RAZORPAY")
  gatewayOrderId String? @unique

```

```

gatewayPaymentId    String?  @unique
gatewaySignature    String?
idempotencyKey      String    @unique
isSandbox           Boolean   @default(true)
failureReason       String?
metadataJson        String?
createdById         String
createdAt           DateTime @default(now())
updatedAt           DateTime @updatedAt
student             Student   @relation(fields: [studentId], references: [id])
studentFee          StudentFee @relation(fields: [studentFeeId], references: [id])
createdBy           User      @relation(fields: [createdById], references: [id])
receipt            PaymentReceipt?

@@index([studentId, status])
@@index([createdAt, status])
}

model PaymentReceipt {
  id          String    @id @default(cuid())
  paymentId   String    @unique
  receiptNumber String    @unique
  studentName String
  studentIdentifier String
  className   String?
  amountPaise Int
  currency    String    @default("INR")
  feeDescription String
  status      String
  issuedAt    DateTime @default(now())
  payment     Payment   @relation(fields: [paymentId], references: [id])
}

model TestRecord {
  id          String    @id @default(cuid())
  title       String
  className   String
  subject     String
  topic       String
  date        String
  time        String
  instructions String?
  status      String    @default("UPCOMING")
  createdById String?
  teacherId   String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  createdBy   User?     @relation("TestCreator", fields: [createdById], references: [id])
  teacher     Teacher?  @relation(fields: [teacherId], references: [id])

  @@index([className, subject])
}

model TeacherAssignment {
  id          String    @id @default(cuid())
  teacherId   String
  className   String
  subject     String
  batch       String?
  createdAt   DateTime @default(now())
  teacher     Teacher   @relation(fields: [teacherId], references: [id])
}

```

```

    @@unique([teacherId, className, subject, batch])
    @@index([className, subject])
}

model StudentEnrollment {
  id          String    @id @default(cuid())
  studentId   String
  className   String
  subject     String
  batch       String?
  createdAt   DateTime  @default(now())
  student     Student   @relation(fields: [studentId], references: [id])

  @@unique([studentId, className, subject, batch])
  @@index([className, subject])
}

model Notification {
  id          String    @id @default(cuid())
  userId      String
  type        String
  title       String
  message     String
  isRead      Boolean   @default(false)
  createdAt   DateTime  @default(now())
  user        User      @relation(fields: [userId], references: [id])

  @@index([userId, isRead])
}

model Course {
  id          String    @id @default(cuid())
  name        String
  slug        String    @unique
  category    String
  classLevel  String
  description  String?
  isActive    Boolean   @default(true)
  subjects    String
  highlights   String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

model Subject {
  id          String    @id @default(cuid())
  name        String
  classLevel  String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

model Batch {
  id          String    @id @default(cuid())
  name        String
  label       String?
  startTime   String
  endTime     String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

```

```

model Timetable {
  id          String    @id @default(cuid())
  day         String
  classLevel  String
  subject     String?
  teacher     String?
  batch       String?
  room        String?
  teacherId   String?
  startTime   String
  endTime     String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  assignedTeacher Teacher? @relation(fields: [teacherId], references: [id])
}

model Facility {
  id          String    @id @default(cuid())
  title       String
  description  String
  image       String?
  isActive    Boolean   @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Enquiry {
  id          String    @id @default(cuid())
  studentName String
  parentName  String
  phone       String
  email       String
  className   String
  subject     String
  batch       String
  message     String
  status      String    @default("NEW")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model AcademySettings {
  id          String    @id @default(cuid())
  siteName    String    @default("Manish Mishra Academy")
  tagline     String    @default("Learn Better. Think Smarter. Achieve More.")
  description  String?
  phone       String?
  email       String?
  address     String?
  whatsapp    String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Testimonial {
  id          String    @id @default(cuid())
  name        String
  role        String?
  quote       String
  isActive    Boolean   @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

```

```

}

===== server/prisma/seed.ts =====
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcryptjs';

const prisma = new PrismaClient();

async function main() {
  const adminPassword = await bcrypt.hash('Mma@2026', 10);

  await prisma.user.upsert({
    where: { email: 'admin@mmacademy.com' },
    update: {},
    create: {
      email: 'admin@mmacademy.com',
      password: adminPassword,
      fullName: 'MMA Admin',
      role: 'ADMIN',
    },
  });

  await prisma.academySettings.upsert({
    where: { id: 'academy-settings' },
    update: {},
    create: {
      id: 'academy-settings',
      siteName: 'Manish Mishra Academy',
      tagline: 'Learn Better. Think Smarter. Achieve More.',
      description: 'Placeholder academy details: update in admin dashboard when official information is available.',
      phone: '+91 00000 00000',
      email: 'info@mmacademy.com',
      address: 'Add academy address here',
      whatsapp: '+910000000000',
    },
  });

  const courses = [
    { name: 'Class 7', slug: 'class-7', category: 'Classes 7-10', classLevel: 'Class 7', subjects: 'Mathematics, Science, English, Social Science, Hindi', highlights: 'Concept building, Practice, Homework, Regular tests, Doubt solving, Faculty guidance' },
    { name: 'Class 8', slug: 'class-8', category: 'Classes 7-10', classLevel: 'Class 8', subjects: 'Mathematics, Science, English, Social Science, Hindi', highlights: 'Concept building, Practice, Homework, Regular tests, Doubt solving, Faculty guidance' },
    { name: 'Class 9', slug: 'class-9', category: 'Classes 7-10', classLevel: 'Class 9', subjects: 'Mathematics, Science, English, Social Science, Hindi', highlights: 'Concept building, Practice, Homework, Regular tests, Doubt solving, Faculty guidance' },
    { name: 'Class 10', slug: 'class-10', category: 'Classes 7-10', classLevel: 'Class 10', subjects: 'Mathematics, Science, English, Social Science, Hindi', highlights: 'Concept building, Practice, Homework, Regular tests, Doubt solving, Faculty guidance' },
    { name: 'Physics', slug: 'physics', category: 'Classes 11-12', classLevel: 'Class 11 / Class 12', subjects: 'Physics', highlights: 'Conceptual understanding, Numerical problem solving, Doubt support, Regular practice' },
    { name: 'Chemistry', slug: 'chemistry', category: 'Classes 11-12', classLevel: 'Class 11 / Class 12', subjects: 'Chemistry', highlights: 'Concepts, reactions, problem solving, regular practice, doubt support' },
    { name: 'Mathematics', slug: 'mathematics', category: 'Classes 11-12', classLevel: 'Class 11 / Class 12', subjects: 'Mathematics', highlights: 'Concept building, problem solving, regular practice, faculty guidance' },
  ];

  for (const course of courses) {
    await prisma.course.upsert({
      where: { slug: course.slug },

```

```

    update: {},
    create: {
      ...course,
      description: 'Placeholder course description for ${course.name}. Replace with official academy details when available.',
    },
  });
}

const batches = [
  { name: 'Batch A', label: '4:00 PM - 5:00 PM', startTime: '16:00', endTime: '17:00' },
  { name: 'Batch B', label: '5:00 PM - 6:00 PM', startTime: '17:00', endTime: '18:00' },
  { name: 'Batch C', label: '6:00 PM - 7:00 PM', startTime: '18:00', endTime: '19:00' },
];

for (const batch of batches) {
  await prisma.batch.upsert({
    where: { id: batch.name },
    update: {},
    create: {
      id: batch.name,
      ...batch,
    },
  });
}

const facilities = [
  { title: 'Smart Classroom', description: 'Modern interactive classroom setup and teaching support.', image: 'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?auto=format&fit=crop&w=1200&q=80' },
  { title: 'AC Classroom', description: 'Comfortable classroom environment that supports focus and learning.', image: 'https://images.unsplash.com/photo-1503676260728-1c00da094a0b?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Library', description: 'Reading and revision area for independent study and academic support.', image: 'https://images.unsplash.com/photo-1521587760476-6c12a4b040da?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Fibre Internet', description: 'High-speed fibre connectivity for educational online study during free time.', image: 'https://images.unsplash.com/photo-1516321318423-f06f85e504b3?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Quiet Study Environment', description: 'A peaceful and disciplined environment for concentration.', image: 'https://images.unsplash.com/photo-1497633762265-9d179a990aa6?auto=format&fit=crop&w=1200&q=80' },
  { title: 'Learning Resources', description: 'Study materials and learning support for concept building and revision.', image: 'https://images.unsplash.com/photo-1454165804606-c3d57bc86b40?auto=format&fit=crop&w=1200&q=80' },
];

for (const facility of facilities) {
  await prisma.facility.create({ data: facility });
}

for (const testimonial of [
  { name: 'Demo Parent', role: 'Parent', quote: 'A welcoming environment with strong focus on concept clarity and discipline.', isActive: true },
  { name: 'Demo Student', role: 'Student', quote: 'The academy creates a calm study atmosphere and helps us understand difficult topics better.', isActive: true },
]) {
  await prisma.testimonial.upsert({
    where: { id: testimonial.name },
    update: {},
    create: {
      id: testimonial.name,
      ...testimonial,
    },
  });
}

```

```

    for (const timetable of [
      { day: 'Monday', classLevel: 'Class 7', subject: 'Placeholder subject', teacher: 'Placeholder teacher', batch: 'Batch A', room: 'Room 1', startTime: '16:00', endTime: '17:00' },
      { day: 'Tuesday', classLevel: 'Class 8', subject: 'Placeholder subject', teacher: 'Placeholder teacher', batch: 'Batch B', room: 'Room 2', startTime: '17:00', endTime: '18:00' },
      { day: 'Wednesday', classLevel: 'Class 9', subject: 'Placeholder subject', teacher: 'Placeholder teacher', batch: 'Batch C', room: 'Room 3', startTime: '18:00', endTime: '19:00' },
    ]) {
      await prisma.timetable.create({ data: timetable });
    }

    console.log('Seed completed');
  }

main()
  .catch((error) => {
    console.error('Seed error:', error);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });

===== server/prisma/seed-part2.ts =====
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcryptjs';

const prisma = new PrismaClient();

async function main() {
  const headPassword = await bcrypt.hash('DevHead@123', 10);
  const supervisorPassword = await bcrypt.hash('DevSupervisor@123', 10);
  const teacherPassword = await bcrypt.hash('DevTeacher@123', 10);
  const studentPassword = await bcrypt.hash('DevStudent@123', 10);

  const headUser = await prisma.user.upsert({
    where: { email: 'head.dev@mmacademy.local' },
    update: {},
    create: {
      email: 'head.dev@mmacademy.local',
      fullName: 'Manish Mishra',
      password: headPassword,
      role: 'HEAD',
      status: 'ACTIVE',
      isActive: true,
    },
  });

  const supervisorUser = await prisma.user.upsert({
    where: { email: 'supervisor.dev@mmacademy.local' },
    update: {},
    create: {
      email: 'supervisor.dev@mmacademy.local',
      fullName: 'Development Supervisor',
      password: supervisorPassword,
      role: 'SUPERVISOR',
      status: 'ACTIVE',
      isActive: true,
      createdBy: headUser.id,
    },
  });
}

```

```

for (const permission of [
  'CREATE_STUDENT',
  'EDIT_STUDENT',
  'CREATE_TEACHER',
  'EDIT_TEACHER',
  'MANAGE_CLASSES',
  'MANAGE_TIMETABLE',
  'MANAGE_RESOURCES',
  'MANAGE_TESTS',
  'MANAGE_ANNOUNCEMENTS',
  'MANAGE_SETTINGS',
]) {
  await prisma.supervisorPermission.upsert({
    where: { userId_permission: { userId: supervisorUser.id, permission } },
    update: { isActive: true },
    create: { userId: supervisorUser.id, permission, grantedBy: headUser.id, isActive: true },
  });
}

await prisma.supervisorPermission.deleteMany({
  where: { userId: supervisorUser.id, permission: { in: ['VIEW_PAYMENTS', 'MANAGE_PAYMENTS'] } },
});

const teacherUser = await prisma.user.upsert({
  where: { email: 'teacher.dev@mmacademy.local' },
  update: {},
  create: {
    email: 'teacher.dev@mmacademy.local',
    fullName: 'Development Teacher',
    password: teacherPassword,
    role: 'TEACHER',
    status: 'ACTIVE',
    isActive: true,
    createdBy: headUser.id,
  },
});

const studentUser = await prisma.user.upsert({
  where: { email: 'student.dev@mmacademy.local' },
  update: {},
  create: {
    email: 'student.dev@mmacademy.local',
    fullName: 'Development Student',
    password: studentPassword,
    role: 'STUDENT',
    status: 'ACTIVE',
    isActive: true,
    createdBy: headUser.id,
  },
});

await prisma.teacher.upsert({
  where: { userId: teacherUser.id },
  update: {},
  create: {
    userId: teacherUser.id,
    teacherId: 'teacher-dev-001',
    classes: 'Class 9, Class 10',
    subjects: 'Mathematics, Physics',
    batches: 'Batch A, Batch B',
    accountStatus: 'ACTIVE',
  },
});

```

```

    },
  });

  await prisma.student.upsert({
    where: { userId: studentUser.id },
    update: {},
    create: {
      userId: studentUser.id,
      studentId: 'student-dev-001',
      className: 'Class 9',
      batch: 'Batch A',
      subjects: 'Mathematics, Science',
      guardianName: 'Development Guardian',
      guardianPhone: '9999999999',
      accountStatus: 'ACTIVE',
    },
  });

  const seededTeacher = await prisma.teacher.findUnique({ where: { userId: teacherUser.id } });
  const seededStudent = await prisma.student.findUnique({ where: { userId: studentUser.id } });
  if (seededTeacher && seededStudent) {
    await prisma.teacherAssignment.deleteMany({ where: { teacherId: seededTeacher.id } });
    for (const assignment of [
      { className: 'Class 9', subject: 'Mathematics', batch: 'Batch A' },
      { className: 'Class 10', subject: 'Mathematics', batch: 'Batch A' },
    ]) {
      await prisma.teacherAssignment.create({ data: { teacherId: seededTeacher.id, ...assignment } });
    }
    await prisma.studentEnrollment.upsert({
      where: { studentId_className_subject_batch: { studentId: seededStudent.id, className: 'Class 9', subject: 'Mathematics', batch: 'Batch A' } },
      update: {},
      create: { studentId: seededStudent.id, className: 'Class 9', subject: 'Mathematics', batch: 'Batch A' },
    });
  }

  await prisma.announcement.createMany({
    data: [
      { title: 'Development Mode Enabled', message: 'These are development-only accounts for testing the MMA role system.', audience: 'ALL' },
      { title: 'Upcoming test', message: 'A class test has been scheduled for next week.', audience: 'STUDENT' }
    ],
  });

  await prisma.testRecord.createMany({
    data: [
      { title: 'Unit test - Mathematics', className: 'Class 9', subject: 'Mathematics', topic: 'Number systems', date: '2026-09-20', time: '10:00', status: 'UPCOMING' },
      { title: 'Mock test - Physics', className: 'Class 11', subject: 'Physics', topic: 'Motion', date: '2026-09-25', time: '10:00', status: 'UPCOMING' },
    ],
  });

  await prisma.feeRecord.createMany({
    data: [
      { studentId: 'student-dev-001', studentName: 'Development Student', amount: '2500', dueDate: '2026-09-30', status: 'PENDING', paymentStatus: 'NOT_PAID' },
    ],
  });

  console.log('Development role seed completed');

```

```
}  
  
main().catch((error) => {  
  console.error(error);  
  process.exit(1);  
}).finally(async () => {  
  await prisma.$disconnect();  
});
```